

Program overview

12-Sep-2023 12:00

Year 2023/2024
Organization Electrical Engineering, Mathematics and Computer Science
Education Bachelor Computer Science and Engineering

Code	Omschrijving	ECTS	p1	p2	p3	p4	p5
B-TI 2023							
Bachelor 1st year TI 2023							
CSE1000	Mentoring	0	✓	✓	✓	✓	✓
CSE1100	Object-oriented programming	5	✓	✓	✓	✓	✓
CSE1105	OOP Project	5	✓	✓	✓	✓	✓
CSE1110	Software Quality and Testing	5	✓	✓	✓	✓	✓
CSE1200	Calculus	5	✓	✓	✓	✓	✓
CSE1205	Linear Algebra	5	✓	✓	✓	✓	✓
CSE1210	Probability Theory and Statistics	5	✓	✓	✓	✓	✓
CSE1300	Reasoning and Logic	5	✓	✓	✓	✓	✓
CSE1305	Algorithms and Data Structures	5	✓	✓	✓	✓	✓
CSE1400	Computer Organisation	5	✓	✓	✓	✓	✓
CSE1405	Computer Networks	5	✓	✓	✓	✓	✓
CSE1500	Web and Database Technology	5	✓	✓	✓	✓	✓
CSE1505	Information and Data Management	5	✓	✓	✓	✓	✓
Bachelor 2nd year TI 2023							
CSE2000	Software Project	15	✓	✓	✓	✓	✓
CSE2115	Software Engineering Methods	5	✓	✓	✓	✓	✓
CSE2120	Concepts of Programming Languages	5	✓	✓	✓	✓	✓
CSE2215	Computer Graphics	5	✓	✓	✓	✓	✓
CSE2220	Signal Processing	5	✓	✓	✓	✓	✓
CSE2225	Image Processing	5	✓	✓	✓	✓	✓
CSE2230	Multimedia Analysis	5	✓	✓	✓	✓	✓
CSE2310	Algorithm Design	5	✓	✓	✓	✓	✓
CSE2315	Automata, Computability and Complexity	5	✓	✓	✓	✓	✓
CSE2420	Digital Systems	5	✓	✓	✓	✓	✓
CSE2425	Embedded Software	5	✓	✓	✓	✓	✓
CSE2430	Operating Systems	5	✓	✓	✓	✓	✓
CSE2510	Machine Learning	5	✓	✓	✓	✓	✓
CSE2520	Big Data Processing	5	✓	✓	✓	✓	✓
CSE2525	Data Mining	5	✓	✓	✓	✓	✓
CSE2530	Computational Intelligence	5	✓	✓	✓	✓	✓
Bachelor 3rd year TI 2023							
CSE3000	Research Project	15	✓	✓	✓	✓	✓
CSE3100	Functional Programming	5	✓	✓	✓	✓	✓
CSE3130	Introduction to Quantum Computer Science	5	✓	✓	✓	✓	✓
CSE3210	Collaborative Artificial Intelligence	5	✓	✓	✓	✓	✓
CSE3220	Computer Security	5	✓	✓	✓	✓	✓
CSE3230	Algebra and Cryptography	5	✓	✓	✓	✓	✓
CSE3300	Algorithms for NP-Hard problems	5	✓	✓	✓	✓	✓
CSE3500	Human Computer Interaction	5	✓	✓	✓	✓	✓

Year	2023/2024
Organization	Electrical Engineering, Mathematics and Computer Science
Education	Bachelor Computer Science and Engineering

Contact for students

Please note! The answers to a lot of your questions can be found on the internet <https://www.tudelft.nl/en/student/>.

Are you a first year student and did you not find the answer to your question? Contact one of your mentors first.

For students that are not in their first year, the first person to contact in case you have any questions, is the commissioner of education of CSE (coi@ch.tudelft.nl). They know the answers to a lot of questions and can also point you towards other people in the faculty or check with them in case of any problems.

During the entire study programme there are General Assemblies: in the first year each quarter and for the higher years each semester. The meetings will be announced on Brightspace and are also scheduled in your timetable.

For questions regarding help and support for personal problems or study delay all students can contact the academic counsellors: <https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/organisation/academic-counsellors/>.

For answers regarding questions about specific courses, first go to the Brightspace page of the course and if you can't find the answer there, contact the responsible lecturer.

Would you like to participate in improving the study programme? Sign up for the so called lecture-response-group (collegeresponsiegroep) of your year. Contact the commissioner of education for more information.

EWI

In association with the Faculty of**Term of validity**

Please note! The information provided here is informative and not binding. Binding information can be found in the regulations <https://www.tudelft.nl/en/student/legal-position/regulations/>.

Program Goals

1. The CSE programme aims to:

educate students so that they can obtain a Bachelor of Science degree in Computer Science & Engineering
enable students to gain admission to one of the Masters programmes following on directly from the Bachelors programme, including the Masters programmes in Computer Science, Computer and Embedded Systems Engineering and Science Education & Communication.

Once you have finished the programme Computer Science & Engineering (CSE) the graduate:

has knowledge of the core concepts and basic methods of the field of computer science such as programming, software engineering, logic, fundamental computer science, databases, web technology, computer systems and networks, information systems and artificial intelligence;

has the necessary mathematical knowledge, such as a knowledge of calculus, linear algebra, probability theory and statistics; is aware of the role of computer science in its application fields;

can apply the above-mentioned knowledge to problems of limited complexity in the field of computer science and its application fields.

has been introduced to research and in particular to modelling in the field of computer science, i.e.

- can analyse problems in order to produce a research plan;
- can use scientific literature to find solutions to studied problems;
- can adapt and implement these solutions.

has a systematic approach in designing software and software systems:

can formulate the requirements to be fulfilled by software and software systems;

can take and substantiate design decisions, taking into account technical, ethical preconditions and socio-economic consequences;

can effectively model the aspects involved;

can choose and apply a suitable software engineering method in order to implement and test that system.

is inquisitive and has an attitude of lifelong learning;

has a systematic approach, e.g. in hypothesis driven problem solving or engineering software;

has the knowledge and the skills to use, justify and assess concepts and theories in computer science as to their value for research and can adapt these for his or her own use;

has insight into the scientific practice in computer science (research system, relation with clients, publication system, importance of integrity, etc.);

knows the methods and techniques used in the scientific practice of computer science and can apply them under supervision; is aware of the limitations of science.

can adopt a critical approach towards and deliberate on his or her own arguments as well as those of others in order to subsequently arrive at a well-founded position;

can recognize and use reasoning methods, ask the right questions, and make and understand qualitative and quantitative statements in the field of computer science.

can work in a professional manner within an international team to solve complex software design problems with fellow computer scientists or others;

can present the results of the work both orally and in writing to fellow computer scientists or others*;

can analyze and discuss the socio-economic, ethical and legal consequences that ill considered, incorrect or poorly designed systems can have*;

can account for a developed computer science artefact (software, algorithm, database, system, etc.) with respect to the responsible use of data and algorithms and the software development process followed.

* In both English and Dutch for the Bilingual track

Program Structure 1

The duration of the study programme in CSE is three years (3 x 60 EC). The first year consists of 12 courses of 5 EC each: 11 regular courses and 1 project. The second year consists of 11 courses: 6 regular courses of 5 EC, a so called Variant (= three consecutive courses) of 15 EC and a large project of 15 EC. The third year is different from the first two years. The first semester you will follow a minor (see below) and 3 elective courses of each 5 EC. The last part of the programme consists of the CSE3000 Research project of 15 EC.

Each year is divided into four quarters with a length of 10 weeks and during each quarter you have three courses of 5 EC in parallel. It is expected that during these 10 weeks you spend 40 hours studying per week. This means that on average you have to study around 13 hours for each course, each week. Not all the hours are planned in your timetable and you will need to do a lot of independent planning and studying.

Program Structure 2

CSE students and their professors are part of the academic community of the TU Delft. Each community has rules of behaviour and the TU Delft has laid them down in the Code of Ethics: <https://www.tudelft.nl/en/student/legal-position/education-regulations/code-of-ethics>. The following rules are from this code:

Professors and students:

- Have respect for others;
- Are involved and transparent and have an honest attitude;
- Contribute to an inspiring work and study environment;
- Trust each other and avoid conflicts of interest;
- Respect each others belongings and resources and those of the university;

	Professors - Have a fair and respectful behaviour towards each other and the students; - Aim for the highest quality and always try to improve; Students - Have a fair and respectful behaviour towards each other and the university employees; - Try to get the most out of themselves by actively taking part in the study program and extracurricular activities; - Stimulate each other and their professors by asking analytical questions and having discussions with good arguments
Exam requirements	The exam requirements specify under which circumstances you are eligible for a CSE diploma. You can find them in the Rules & Regulations of the Board of Examiners https://www.tudelft.nl/en/student/eemcs-student-portal/education/rules-regulations. In summary, it is stated that you can receive your CSE diploma when you have passed all courses, included the minor, with a grade of 6 or higher.
With Honours Regulation	Once you have finished the Bachelor programme you can request your diploma with a form on the following website: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/forms/. The requirements for receiving a diploma "with distinction" can be found in the Rules and Regulations of the Board of Examiners: https://www.tudelft.nl/en/student/eemcs-student-portal/education/rules-regulations
Administration by the Faculty of	EWI
Minor Semester	In the first semester of your third year you will follow a minor. The minor gives you the possibility to broaden your knowledge or to go abroad and gain experience in living and studying in another country.
Expected prior Knowledge	More information can be found on https://www.tudelft.nl/en/education/programmes/minor/. Students can prepare for the first semester of the first year by: - Doing this course on programming in Java: https://www.codecademy.com/learn/learn-java - Brushing up your calculus skills by joining the Pre-university Calculus MOOC: https://www.edx.org/course/pre-university-calculus-2
Education Methods	Lecturers of the CSE programme use many different education methods, depending on the course material that has to be taught. The most frequently used education methods are listed below: - Lecture: the lecturer gives a presentation about the material in a lecture room. Often the lectures are complemented with homework assignments. - Instructions: during instructions you can work on assignments with the help of lecturers and teaching assistants. The groups are kept small so you have the opportunity to ask questions. - Interactive lecture: an interactive lecture is a combination between a lecture and an instruction. The professor explains material regarding assignments while in between the explanations you get the chance to work on the assignments. - Practical/lab: during a practical or lab you work (mostly independently) on programming assignments and practical problems with the help of teaching assistants. - Project: a project is a practical assignment you solve with a group. Usually, at the end of a project a deliverable has to be handed in. Teamwork is an important part of a project. - Independent study: For all the discussed methods mentioned above it is essential that you are the one doing the work. Only this way you will be able to apply the things that you have learned by yourself.
Honours track	For students who are looking for something more challenging it is possible to join the Honours Programme Bachelor (HPB) in the second year. Students who have finished all the courses of the first year within one year with an average grade of at least an 8.0 are eligible to apply for the Honours Programme.

Year	2023/2024
Organization	Electrical Engineering, Mathematics and Computer Science
Education	Bachelor Computer Science and Engineering

Bachelor 1st year TI 2023	
Prerequisites	Please note! The entry requirements are mentioned in the regulations https://www.tudelft.nl/en/student/eemcs-student-portal/education/rules-regulations . If there are differences between what is mentioned here and what is mentioned in the regulations, that what is mentioned in the regulations is binding.
Introduction 1	Registration for exams For the exams of all the courses you have to register at least two weeks before the day that they take place. More information is available on: https://www.tudelft.nl/en/student/eemcs-student-portal/education/registration-courses-and-examinations. If you decide after your register that you dont want to participate in the course or exam dont forget to cancel your registration.
Introduction 2	As is the case for all the programmes of the TU Delft, also for CSE you must obtain at least 45 EC in your first year. This is called the binding recommendation on continuation of studies (BSA). For more information go to: https://www.tudelft.nl/en/student/legal-position/regulations/.
Introduction 3	Preview year 2: In the last quarter of the first year you will choose a so called Variant that you will follow during the first three quarters of your second year. More information on the Variants can be found on the study guide pages about the second year and will be shared during the General Assemblies.

CSE1000	Mentoring	0
Responsible Instructor	P. Pawelczak	
Contact Hours / Week x/x/x/x	1/1/0/0	
Education Period	1 2	
Start Education	1 2	
Exam Period	1 2	
Course Language	English	
Course Contents	Students are divided into groups with accompanying student and teacher mentor. These mentor groups meet in the first semester one time per week (around the lunch time) and discuss important issues such as time management, learning to study, fraud, exams, etc. Additionally, depending on choice, individual meetings with each student are performed.	
Study Goals	Goals of the Mentoring: 1. Support social bonding within the students group and the university 2. Tighten the relationship between students and the computer science programme at TU Delft 3. Contribute to an inclusive learning environment 4. Raise the awareness of a students studying attitude 5. Help students to improve their study methods 6. Help in self-reflecting on students study progress 7. Expand the students knowledge of study process and university-related matters like academic fraud or BSA	
Education Method	Weekly mentor group meetings in Semester 1 and individual face to face meetings with each student.	
Assessment	By means of verifying obligatory presence on the mentor meetings and the completion of the assignments associated with the Mentoring. Students who have missed too many mentoring sessions are offered a repair assignment at the end of Q2. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	J.E. van der Laan	
Co-Instructor	J.J. Koorn	

CSE1100	Object-oriented programming	5
Responsible Instructor	Ir. T.A.R. Overklift Vaupel Klein	
Contact Hours / Week x/x/x/x	6/0/0/0 lecture; 4/0/0/0 shared lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Course Contents	Introductory programming course focused on object oriented programming in Java. The course covers: - Primitive datatypes: value domains, operators, expression, declarations and assignments. - Control structures: conditional selection, repetition. - Composition of data: arrays, Strings, ArrayLists, Sets. - Classes: specification, implementation, equality & hashcodes. - Inheritance: modeling inheritance, type casts, polymorphism, static and dynamic binding. - Use of libraries (APIs). - Code quality: best practices, unit testing (with JUnit), debugging strategies. - Exception handling. - Generics. - Basic IO operations (user input / reading / writing). - Design & implementation of small programs. - Functional programming in Java (streams, map, filter).	
Study Goals	After completing this course, the student: - Can describe the key characteristics of object-oriented programming. - Can write methods and classes using the Java programming language. --- Can write methods and classes using correct indentation and coding style. --- Can write an equals method for a class using the Java programming language. --- Can create and manipulate a data structure using methods and classes. - Can design, implement and debug a simple program that contains inheritance and interfaces using the Java programming language. - Can design, implement and debug a simple program that reads from files, and writes to files using the Java programming language. - Can write unit tests for a program in Java using the Junit testing framework. - Can effectively search for existing packages, classes and programming solutions, using the Java documentation as well as online resources, and apply this information in their own programs.	
Education Method	Lectures, tutorials, and practical sessions with programming exercises. The programming exercises are feedback assignments, i.e., they do not count towards your grade, but you can get feedback on them to improve your programming skills.	
Assessment	This course contains two summative exams: 1) Theoretical exam in Weblab (MC, Open & (Small) Programming questions; no auxiliary materials may be used). 2) Programming exam (Open book, IDE & additional resources/documentation allowed). This course contains two formative tests. These do not count towards your final grade, but allow you to practice programming in a real exam environment: 1) A Midterm Theoretical Exam (in week 5). 2) A Mock Programming Exam (in week 8). Between the two formative tests you can score a bonus of at most 0.5 points on your final grade. For both the Midterm and the Mock Exam you get a score of at most 0.5 bonus points. Your final bonus is calculated as: MIN(0.5, Midterm Bonus + Mock Exam Bonus), this means the bonus on top of your final grade is capped at 0.5 points. Your final grade is composed of: 1) The Theoretical Exam (weight 0.5, minimal required grade 5/10) 2) The Programming Exam (weight 0.5, minimal required grade 5/10) If your score for the Theoretical Exam and Programming Exam are (1) both minimally a 5/10 and (2) the average of both is at least 5.8, you pass this course. If you scored any bonus for the formative tests, this final bonus with ONLY be applied if you pass the course based on your grades for the Theoretical Exam and Programming Exam. You CANNOT compensate a failing grade with your bonus points. Within the academic year you can resit the Theoretical Exam and the Programming Exam in quarter 2. Out of the two attempts for the Theoretical Exam and the Programming Exam respectively, the highest score will count towards your final grade. Partial grades for the Theoretical Exam or Programming Exam cannot be carried over to a next academic year. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Special Information	This course makes use of a course email: oop-cs-ewi@tudelft.nl	
Co-Instructor	Prof.dr. A.E. Zaidman	

CSE1105	OOP Project	5
Responsible Instructor	Dr.ing. S. Proksch	
Contact Hours / Week x/x/x/x	0/0/2/0 lecture; 0/0/1/0 lab	
Education Period	3	
Start Education	3	
Exam Period	3	
Course Language	English	
Course Contents	Modern software development is not just about knowing the right programming language or the right technology stack. It is as important to know how to work together in a team to get a serious programming project done. In this course, you will learn how to collaboratively create a software system. The contents of this course include learning about teamwork, interpreting requirements given by a client and breaking them down into actionable tasks, using popular collaboration tools like GitLab to plan and track project progress, using collaborative tools like Git, and learning popular frameworks and libraries to bring the project to success. The first weeks of the course will be structured through lectures to get you and your group started, afterwards, the course will transition into the implementation phase, in which you will work on the project. Every group will be closely supervised by a tutor. *Important:* The course has mandatory presence from week 1 onwards. Plan your travel to Delft accordingly.	
Study Goals	After completing this course, you will be able to ... 1) Act professionally in a team of software developers ... Get insight into how you work in a team and make agreements ... Chair a meeting and take minutes ... Give and receive feedback 2) Manage a complex software project in a group ... Break down a formal product backlog into actionable tasks ... Plan and manage the project tasks in GitLab ... Organize and distribute tasks between team members 3) Follow a collaborative software engineering process ... Use Git to version and share source code contributions ... Contribute source code to a group project through GitLab merge requests ... Evaluate code contributions of other team members in GitLab merge requests 4) Create a client/server application from a provided set of project requirements ... Build a server application with Spring Boot ... Build a graphical client with Java FX ... Exchange data through REST and web sockets using an object mapper like Jackson ... Implement reactive REST clients with long-polling ... Manage the lifecycle of an application through dependency injection ... Write unit tests for a complex, composed software system 5) Apply usability principles in a graphical user interface	
Education Method	Following this course will require you to... Follow interactive lectures and pre-recorded material for self-study Participate in a collaborative programming project in a small team Assess the work of yourself and others through merge requests (code) and peer reviews (process) Actively participate in TA sessions Present your project results and answer questions about the lecture contents.	
Assessment	--- Formative Assessment --- You can check your individual learning progress through weekly rubrics ... You can self-assessment your own performance with respect to the rubrics ... You will receive feedback from a TA as a neutral outsider You will discuss your individual learning progress and success in weekly TA meetings --- Individual Knock-Out Elements --- The course enforces several criteria to protect the active members of a team. Students not reaching the following criteria will be removed from the course: Uphold a professional attitude when interacting with peers and staff. You cannot miss more than one TA meeting without approval. Absence will only be approved on a case-by-case basis, when a valid reason has been indicated to the lecturers(!) before(!) missing a meeting. Active participation in the team process and contributing to all team deadlines and deliverables Meaningful and visible project contributions throughout each of the seven project weeks (W2-4, W6-9) ... 100+ Lines of Java source code (i.e., no comments, whitespace, JavaFX XML, documents, ...) ... 3+ Commits of Java source code ... At least one of your merge-requests gets accepted by your team ... You add a constructive comment on at least one merge request of someone else (100+ characters) Ability to explain lecture contents and defend all aspects of the team project in the oral examination We offer repair options for the previous two criteria: Not having meaningful and visible contributions in one week can be repaired in the following week. Failing to satisfy the criteria in more than one week will lead to removal. The inability to explain or justify the project in the oral examination can be repaired in a second oral examination. --- Summative Assessment --- Students attending the course will be graded on the following grading components. ALL graded components are rounded to .1 and must at least be "almost passed" (>5.0). Their weighted average will be rounded to .5 to calculate the final team grade.	

Process:
 ... Tasks and Planning (25%)
 ... Code contribution and code reviews (25%)
 Product:
 ... Git Assignment (Pass/Fail, Individual)
 ... Implemented Features (25%)
 ... Technology (10%)
 ... Testing (10%)
 ... Usability (5%)
 Teamwork (All components are Pass/Fail):
 ... Code of Conduct
 ... Conducting an Effective Meeting (Individual)
 ... Providing Constructive Peer Feedback with the AID Model (Individual)
 ... Self Reflection (Individual)
 Project-Pitch & Oral Examination (Pass/Fail)

In case of large performance differences in a group, we may choose to assess the team members individually, based on the individual contributions to the deliverables and the interaction traces that we find in the repository.

--- Please Note ---

All assignment deadlines throughout the course are fixed, we do not accept late submissions. Late submissions in systems that do not enforce due dates will be treated as not submitted (e.g., Brightspace, Buddycheck).
 Failing to submit a deliverable cannot be repaired.
 Insufficient deliverables (≥ 4.0 and < 6.0) can be repaired, but the grade of the deliverable is then capped at 6.0.
 There is NO resit opportunity for this course.
 Partial grades are not carried over to the next academic year.

Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)

Enrolment / Application

In addition to your regular enrollment of the course, you must also register in Brightspace, AT THE LATEST during the first lecture, so you can be assigned to a group. Late registrations after the group creation cannot be considered anymore.

Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE program or an EEMCS bridging program (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: <https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/>

Co-Instructor

Prof.dr. A.E. Zaidman

Co-Instructor

I.C. de Bruin

CSE1110	Software Quality and Testing	5
Responsible Instructor	M. Finavaro Aniche	
Contact Hours / Week x/x/x/x	0/0/0/4 lecture; 0/0/0/4 shared lab	
Education Period	4	
Start Education	4	
Exam Period	4 5	
Course Language	English	
Course Contents	The course covers the most important testing techniques needed to build high quality software systems. The course covers both the testing techniques themselves, and the software design techniques needed to create testable systems. Specific topics covered are: * testing principles, test levels, and the testing pyramid * software testing automation and JUnit * specification-based testing, equivalence partitioning and domain testing * boundary analysis and testing * property-based testing * structural testing and the different coverage criteria * mock objects * design for testability * test-driven development (TDD) * test code quality and test smells * larger tests	
Study Goals	By the end of the course, students will be able to: * Create unit, integration, and system tests using current existing tools (i.e., JUnit, Mockito, and JaCoCo) that effectively test complex software systems. * Derive test cases that deal with exceptional, corner, and bad weather cases by performing different testing techniques as well as able to reflect about their limitations, when and when not to apply them in a given context. * Measure and reflect on the efficiency of the developed test suites by means of different test adequacy metrics (i.e., line, branch, condition, MC/DC coverage). * Design and implement testable software systems by means of architectural patterns such as dependency injection and ports and adapters. * Write maintainable test code by avoiding well-known test code smells. * Understand the principles of state-of-the-art techniques for intelligent software testing, such as property-based testing.	
Education Method	This course follows a flipped classroom approach. Students read chapters of the book and work on (practical and theoretical) exercises during their study time; lectures are used for questions and broader discussions. We offer TA support during the labs.	
Books	Effective Software Testing, by Maurício Aniche: https://www.effective-software-testing.com .	
Assessment	The evaluation of the course is based on one exam. It happens on WebLab and is composed of coding questions and theoretical questions. Students can consult the documentation of certain testing libraries during the exam. There is also one possibility to do a resit.	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	F. Mulder	

CSE1200	Calculus	5
Responsible Instructor	N. de Kleijn	
Contact Hours / Week x/x/x/x	0/6/0/0 colstruction; 0/1/0/0 lecture	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Expected prior knowledge	Reasoning & Logic, High school mathematics.	
Course Contents	The main objective of this course is to provide a solid basis of mathematical concepts and skills that will be necessary in the rest of the Bachelor Computer Science & Engineering. The material treated in this course plays an important role in courses such as Algorithms and Data Structures, Linear Algebra, Computer Graphics, Probability and Statistics, Signal Processing and Image Processing.	
Study Goals	Here is a list of topics that will be treated in this course, and the associated learning objectives. Understanding of functions. The student can determine domain, range and inverse of a given function, and sketch its graph. The student can simplify compositions of functions, in particular in case of (inverse) trigonometric functions. Limits. The student knows and understands the concept of limits as well as the associated rules of calculation. The student can evaluate limits. The students knows the conditions under which 'Hospitals Rule can be applied, and can apply the rule in these cases. Differentiation. The student knows the rules of calculation for differentiation, in particular the Chain Rule. The student can apply those to determine derivatives (first order and higher) of a given function, if necessary by implicit differentiation. The student can determine linearisations and differentials of functions. Integration: The student knows and understands the definition of an integral in terms of Riemann sums, and can apply the Fundamental Theorem of Calculus to calculate integrals. The student knows the rules of calculation for integration, in particular the Substitution Rule and Integration by parts, and can employ these to find anti-derivatives and evaluate integrals. Sequences and Series. The student can determine convergence and limits of a (possible recursively defined) sequence. The student masters techniques, in particular the Comparison test, Alternating Series Test and the Ratio Test, to determine convergence and absolute convergence of a given series. The student can determine the radius of convergence and interval of convergence of a given power series. The student is able to determine the limit of certain types of (power) series. Complex numbers. The student knows the rules of calculation for complex numbers and can apply those to solve algebraic equations. The student knows and can apply Euler's Identity and De Moivre's Rule. Multivariate functions. The student can sketch and interpret graphical representations of functions of several variables. The student can determine linearisations, tangent planes, directional derivatives, gradient and local extrema of multivariate functions. Integration of multivariate functions. The student can evaluate the integral of a function in two variables on a simple region and interpret the answer.	
Education Method	Every week, there will be 6 hours of lectures for this course.	
Books	For the course material and exercises we use the book "Calculus: Early Transcendentals', Interational Metric Edition, 9th edition, by James Stewart. ISBN: 978-1-305-27237-8 PLEASE NOTE: the use of older edition may cause problems, since the numbering of exercises and sections varies between editions.	
Assessment	The final grade of this course is determined by two tests; one in week 2.5 and one in week 2.10. There will be one overall retake. It is not possible to do a retake for separate tests. After the final test, the final grade for this course is calculated as follows: $(\text{grade test 1})/3 + 2 * (\text{grade test 2})/3$ The course is passed if this grade is 5.75 or higher AND the grade of the second test is 5 or higher. If the grade for the resit is higher than the final grade, it will replace the final grade. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	N. de Kleijn	

CSE1205	Linear Algebra	5
Responsible Instructor	Dr. E. Emsiz	
Contact Hours / Week x/x/x/x	0/0/6/0 colstruction; 0/0/2/0 lecture	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Required for	Multivariate data Analysis; Computer Graphics; Image Processing	
Course Contents	- Solving systems of linear equations (using Gaussian elimination) - Vectors in $\mathbb{R}^n$ - Linear dependence/independence - Matrix algebra (sum, product, inverse) - LU decomposition of a matrix - Linear transformations - Subspaces in $\mathbb{R}^n$ (basis, dimension); column space and null space - Determinants - Eigenvalues and eigenvectors - Diagonalization - Inner products and orthogonal sets - Orthogonal projections - Method of least squares and linear models - Symmetric matrices	
Study Goals	An important goal of the course is to make the student comfortable with fundamental notions from linear algebra. At the end of this course you will be able to understand the connections between these concepts. You may also be asked to come up with proofs for elementary properties (i.e. write down a consistent/correct short proof) and give a correct counterexample. After successfully completing the course you will be able to: - Solve systems of linear equations. - Identify if a system of linear equations is solvable, and if the solution is unique. - Show the equivalence between a vector equation, a matrix equation and a system of linear equations. - Give a geometric description of solutions sets of vector equations. - Determine the linear (in)dependence of a set of vectors and identify the (geometric) properties of linear dependence. - Identify if a transformation is linear and determine the standard matrix of a linear transformation. - Determine the matrix of geometric linear transformations. - Be able to perform matrix operations (sum, scalar multiple, multiplication, transpose) and describe the law-like properties of matrix multiplication and apply these properties. - Describe the properties of invertible matrices, determine the inverse of a non-singular matrix and apply these topics. - Give definitions of subspace, column space and null space of a matrix and determine a basis for a subspace. - Describe and apply the concept of dimension of a linear subspace, rank of a matrix and the rank theorem. - Determine the LU-decomposition of a matrix. - Determine the determinant of a matrix by applying the properties of a determinant. - Apply the properties of determinants (product, transpose, inverse). - Determine the eigenvalues and eigenvectors of a matrix. - Determine the algebraic and geometric multiplicity of a matrix. - Determine if a matrix is diagonalizable and if so, determine a diagonalization of the matrix. - Analyze the case of complex eigenvalues and eigenvectors. - Determine the matrix of a linear transformation with respect to a basis. - Calculate and apply properties of the inner product in the context of orthogonality and orthogonal sets. - Calculate the orthogonal projection on a subspace. - Determine an orthogonal basis of a subspace by means of the Gram-Schmidt process. - Explain the normal equation of the least-squares method and apply the least-squares method to linear models. - Explain and calculate the orthogonal diagonalization of symmetric matrices and describe the spectral theorem for symmetric matrices.	
Education Method	Lectures blended with exercise sessions, online homework exercises and instructions. Disclaimer: information may change depending on the developments around the coronavirus.	
Literature and Study Materials	Book: David C. Lay et. al., Linear Algebra and its Applications, Global Edition, 6/E. - Slides on Brightspace. - Prelecture videos. - Online homework assignments. - Book exercises - Old exams on Brightspace.	
Assessment	Disclaimer: information may change depending on the developments around the coronavirus. Final score: F IF the midterm score M is higher than the score E for the exam AND $E \geq 5.0$ THEN $F = 0.33 M + 0.67 E$ ELSE $F = E$ All grades are rounded to 1 decimal place. Note that the midterm is not mandatory (and can not be resited). For the resit the midterm is not relevant anymore. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Permitted Materials during Tests	No material.	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	

Co-Instructor	Drs. P.R. van Nieuwenhuizen
----------------------	-----------------------------

CSE1210	Probability Theory and Statistics	5
Responsible Instructor	Dr. C.O. Smet	
Contact Hours / Week x/x/x/x	0/0/0/6 colstruction	
Education Period	4	
Start Education	4	
Exam Period	4 5	
Course Language	English	
Expected prior knowledge	Calculus, in particular differentiating and integrating functions of one and two variables is indispensable.	
Course Contents	Graphical and numerical summaries of data Introduction to probability Some elementary combinatorics to find probabilities Conditional probability; Bayes' rule; stochastic (in)dependence Random variables; probability mass function; density function; distribution function Standard distributions: Binomial, Poisson, Geometric, Normal, Uniform, Exponential, Pareto Expectation and variance; transformations; Jensen's inequality Multivariate random variables; joint distributions; joint and marginal density functions; (in)dependence of random variables Covariance and correlation Chebychev's inequality; Law of Large numbers; Central Limit Theorem Sampling theory and statistical models; mean, sample variance, histogram, empirical distribution function, boxplot Theory of Estimators: Bias, Efficiency, Mean squared error Linear Regression: univariate and multivariate, goodness of fit Introduction to statistical learning Confidence intervals of Mean and Proportion; t-distribution Testing theory: Type I/II Error, p-value, significance level, critical region One sample t-test Parametric bootstrap	
Study Goals	At the end of the course, the students will be able to: - compute (conditional) probabilities - use discrete and continuous random variables - use joint random variables - compute expectation and (co)variance - apply the law of large numbers and the central limit theorem - analyse data by means of graphical and numerical summaries - formulate a statistical model - estimate parameters - perform a linear regression analysis - construct a confidence interval - formulate and carry out hypothesis tests - perform a bootstrap analysis - Use Python to perform elementary simulation and data analysis	
Education Method	Education will take place in a blended way: the student watches a prelecture video, makes some exercises related to this, participates in the lecture (which consist of lecturing by the teacher and making exercises) and then makes some more exercises at home. This cycle is repeated for every lecture.	
Literature and Study Materials	We use the "Blue Book": A Modern Introduction to Probability and Statistics Understanding Why and How Series: Springer Texts in Statistics Dekking, F.M., Kraaikamp, C., Lopuhaä, H.P., Meester, L.E. 2005, XVI, 488 p. 120 illus., Hardcover ISBN: 978-1-85233-896-1	
Assessment	The final grade of the course consists of the following components: - Midterm on Probability Theory (50%) - Endterm on Statistics (50%) - 4 Computer Assignments (CA, for a maximum of one bonus point, this value in [0,1] will be added if $0.5 * \text{Midterm} + 0.5 * \text{Endterm}$ is at least 5) Final grade calculation: $(0.5 * \text{Midterm} + 0.5 * \text{Endterm} + \text{CA})$ Repair possibilities: - Resit in next quarter - CA: re-submit assignment, the repair grade will be capped at 6.0 Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Permitted Materials during Tests	Formula sheet (one sheet, to be made by the student) and a simple, non-programmable, non-graphical calculator.	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Tags	Mathematics Stochastics	
Co-Instructor	N. de Kleijn	

CSE1300	Reasoning and Logic	5
Responsible Instructor	S. Hugtenburg	
Responsible Instructor	Dr. N. Yorke-Smith	
Course Coordinator	S. Hugtenburg	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 4/0/0/0 shared lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Required for	Every course in which exact reasoning is required, e.g. to prove a theorem, or reasoning from the assumption that a theorem is true. Examples: * Calculus * Algorithms and Data Structures * Linear Algebra * Probability and Statistics * Machine Learning * Algorithm Design * Automata, Languages and Computability * Logic-Based Artificial Intelligence * Cryptography * Software Verification * Complexity Theory	
Expected prior knowledge	High school mathematics	
Course Contents	It is often useful or even essential to know if a certain statement is true, e.g. Pythagoras' famous $a^2 + b^2 = c^2$ theorem about right-angled triangles. Knowledge gathered from these statements or theorems can be broadly used to solve more complicated problems. This way of working, i.e., deriving more complex theorems from simpler ones, is useful in many fields, in particular also in computer science. An argument is a set of premises or assumptions, followed by a conclusion. To be certain of the truth of an argument, the conclusion has to be a logical consequence of the assumptions. To prove this, the conclusion is derived from the premisses. The derivation shows us that once all premises are true, the conclusion is true as well. The course Reasoning and Logic is about proving the logical validity of arguments. What is a valid argument? When is an argument logically valid and when is it not? How can we determine whether an argument is logically valid? How can we derive a logically valid conclusion from the premises? Or how can we prove that a conclusion is not a logical consequence of the premises? In this course we will first explain a number of basic proof techniques, such as proof by contradiction, proof by mathematical induction, proof by division into cases, and the use of invariants. The application of these techniques will be practiced by proving and rejecting simple mathematical theorems. These proof techniques can only lead to a valid argument when the formulation of the premises and the conclusion is sufficiently precise. To express statements precisely, multiple artificial languages exist, of which we will learn two: propositional calculus and predicate logic. For both languages we look at the syntax and semantics and study how to translate expressions from a natural language to the more exact languages. Furthermore we will look at how to establish the logical validity of an argument in both languages. Moreover, to be able to assign truth values to formulas in predicate logic and because of the importance of this subject in every exact science, in this course we will also pay attention to elementary set theory. Together, the course provides tools that have important applications across mathematics, computer science, philosophy, and beyond.	
Study Goals	After this course, the student is able to: 1. translate a logically precise claim to and from natural language. 2. describe the operation of logical connectors and quantifiers. 3. describe the notion of logical validity. 4. explain and apply basic set and graph operations. 5. define and perform computations with functions, relations and equivalence classes. 6. construct and interpret recursive definitions, including recursive data structures like trees. 7. construct an appropriate function or relation given a description (in natural language or formal notation). 8. construct a direct or indirect proof (by contradiction, division into cases, generalisation, or [structural] induction) or logical equivalence, or counterexample for (in)valid arguments, in propositional logic, predicate logic and set theory. 9. identify what type of proof is appropriate for a given claim. 10. solve simple SAT instances. 11. develop specifications for verification tools like SAT or SMT solvers. 12. interpret the response of verification tools like SAT or SMT solvers	
Education Method	This course uses a student-centred learning methodology. Students study provided material at home. Every week has two sessions, during which questions about the material will be answered. There will also be small interactive exercises in some of these sessions to help the students to evaluate their understanding of the material and to apply it. Optional homework assignments are provided. These may be done in pairs. Lab sessions are offered, in which answers to the homework questions can be discussed. A project is also included in the course. This must be done individually.	
Books	Delftse Foundations of Computation, Hugtenburg & Yorke-Smith, TU Delft Open 2019, textbooks.open.tudelft.nl, ISBN 978-94-6186-952-4	
Assessment	The final grade of the course is computed from several components, described below. A final exam (100%) in week 10 (F) Additionally there are two opportunities to achieve some bonus points: Teamwork challenge Week 5 (at most 0.5 points) (T) Project, deadline Week 9 (at most 0.5 points) (P) Between S and P, you can score a bonus of at most 0.5 points on your final grade. Your final bonus $B = \min(0.5, T + P)$. Final grade The final grade for the course is computed as follows: If $F < 5.8$, the final grade is F.	

	Else the final grade is $X = \min(F + B, 10)$. The resit (R) covers all material. If $R < 5.8$, the final grade is R. Else the final resit grade $Y = \min(10, R + B)$. As per TU delft regulations, the grade considered for the course is $\max(X, Y)$. Note that these formula also indicate that the bonus cannot turn a failing grade into a passing grade. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)
Permitted Materials during Tests	Course materials available in Brightspace and WebLab, a (physical or digital) copy of the textbook are allowed if the exams _cannot_ take place on campus. Otherwise no materials are allowed during the exam.
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/
Special Information	For any administrative questions about the course, please use rl-cs-ewi@tudelft.nl and not the instructors personal e-mail addresses.
Tags	Abstract Algorithmics Logics Mathematics Project Small groups
Co-Instructor	S. Dumani

CSE1305	Algorithms and Data Structures	5
Responsible Instructor	M. Khosla	
Responsible Instructor	Dr. J.S. de Pinho Gonçalves	
Responsible Instructor	I.D. van Kreveld	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/4/0/0 shared lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Expected prior knowledge	Java programming, constructing proofs, trees and graphs	
Course Contents	Algorithms and data structures are fundamental notions in computer science . Understanding how they can be exploited in combination for better programming implementations in terms of time and space complexity is vital for writing efficient code.This course enables the student to: - Explain and compare the structure and properties of standard algorithms and data structures. - Execute and visualize standard algorithms and data structures on given inputs. - Use mathematical methods to analyze the time and space complexity of algorithms and data structures. - Implement algorithms and data structures using the Java programming language. - Solve programming tasks using standard algorithms and data structures.	
Study Goals	1. Data Structures - data containers -- vector -- list -- tree -- set - ordered data structures -- stack -- queue -- priority queue -- heap -- map - operations on data structures -- iterative implementations -- recursive implementations The student can explain essential data structures, explain and analyze the core properties of these data structures, and explain, develop, and adopt iterative and recursive implementations of operations on these data structures, and analyze and compare space and time complexity of these operations. 2. Sorting - selection sort - insertion sort - heap sort - merge sort - quick sort - bucket sort - radix sort The student can explain and implement various sorting algorithms, analyze and compare space and time complexity of these algorithms, and choose sorting algorithms for different problem scenarios. 3. Searching - search structures -- search trees -- AVL trees -- (2,4) trees - backtracking The student can explain data structures which support efficient search and their invariants, explain, develop, and adopt implementations of operations on these data structures, analyze and compare space and time complexity of these operations, choose search structures for different problem scenarios, and explain, implement, and analyze backtracking algorithms. 4. Graphs and Graph Algorithms - graph data structures -- directed graphs -- undirected graphs -- weights -- representations - graph algorithms -- graph traversals -- path finding -- cycle finding -- connectivity -- topological ordering -- shortest path -- minimum spanning tree The student can explain various kinds of graphs, explain, analyze and compare different data structures for graphs, explain, develop and adopt implementations of operations on these data structures, analyze and compare space and time complexity of these operations, choose graph data structures for different problem scenarios, explain, implement, apply and analyze generic graph traversal strategies, and explain, develop and adopt implementations of various graph algorithms.	
Education Method	Every week has two online sessions during which questions about the material will be answered, and small exercises will be completed with the students. Students will study the relevant material before every session through videos, slides, and the course	

	book. Homework assignments are provided. Lab sessions are offered, in which answers to the homework can be discussed.
Books	Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser Data Structures And Algorithms In Java 6th edition ISBN 978-1-118-80857-3
Assessment	The course comprises the following assessment moments: Week 5: a formative analysis midterm exam (MA) Week 5: a formative implementation midterm exam (MI) Week 10: a summative analysis final exam (FA) Week 10: a summative implementation final exam (FI) The final course grade GF will be as follows: If $FA \geq 5$ and $FI \geq 5$, then $GF = (FA+FI)/2$ Otherwise, $GF = NV$ For successful completion of the course: - both grades FA and FI need to be at least 5 (before rounding), - the final course grade GF needs to be at least 5.8. The standard TU Delft rounding rules apply. Note: the midterms do not contribute to the final course grade, their purpose is only to give the student an idea of the progress halfway through the course. There are separate resits for the analysis (RA) and the implementation (RI) components. The course grade GR after the resit will be determined as follows: If $MAX(FA,RA) \geq 5$ and $MAX(FI,RI) \geq 5$, then $GR = (MAX(FA,RA)+MAX(FI,RI))/2$ Otherwise, $GR = NV$ For successful completion of the course after the resit: - FA or RA needs to be at least 5 and also FI or RI needs to be at least 5 (all before rounding), - the course grade after resit GR needs to be at least 5.8. The standard TU Delft rounding rules apply. Partial grades cannot be carried over to future editions of the course. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)
Permitted Materials during Tests	Paper-based analysis exams: you are allowed to bring a hand-written double sided A4 cheat sheet. No additional materials are permitted, including but not limited to: books, printouts of the slides, or devices. Computer-based implementation exams: no materials are allowed during the computer-based implementation exams.
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/
Co-Instructor	S. Hugtenburg

CSE1400	Computer Organisation	5
Responsible Instructor	Ir. O.W. Visser	
Responsible Instructor	Prof.dr. K.G. Langendoen	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 1/0/0/0 tutorial; 4/0/0/0 shared lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Expected prior knowledge	High-school-level math.	
Course Contents	Rationale Computers are everywhere. Topics include: the architecture, the structure, the operation and the interconnection of computer components into computer systems, including modern architectures, data representation, assembler programming, virtual machines, the structure of translators, compiling and loading, basic operating systems concepts (I/O, interrupt handling, process).	
Study Goals	Students can: 1. Explain the basic concepts, historical objectives, and modern functions of digital computers. 2. Evaluate and recommend number representation systems and digital circuits when given a set of realistic system constraints. 3. Derive conversion functions for Boolean algebra. 4. Use arithmetic and conversion functions on well-known number representation formats. 5. Describe the architecture and operation of digital computers. 6. Perform operations using each of the main components of a digital computer: the basic processing unit, the I/O system, the memory system, the interconnection system. 7. Explain system mechanisms for building faster processors, such as pipelining, memory organization, caching, and parallelism. 8. Analyze the trade-offs inherent in the design of digital computers, concerning among others performance, cost, scalability, availability, and energy consumption. 9. Demonstrate proficiency with basic assembly programming by implementing realistic basic programs used in digital computers.	
Education Method	Lectures, tutorial, and practical work (Lab). Self-study with peer reviewing. This course uses gamification.	
Literature and Study Materials	Course and Lab guides are also provided via Brightspace.	
Books	C. Hamacher e.a., Computer Organization, 6th edition, McGraw-Hill, 2012	
Assessment	(Mandatory) 3 basic lab assignments. Does not award any points (checked by TAs). If you pass 2 out of 3 assignments, you can repair the third (failed) assignment by doing one of the bonuses 5a, 5b, or 5c (NB: this bonus done as repair does not award any points). (Mandatory) exam, written. Mid-term in week 5; second part in week 10. There is a complete resit of both parts in the next quarter. Minimum total exam score: 4000 points. (Voluntary) in-class exercises, oral and written. (Voluntary) self-study of relevant questions with peer feedback. (Voluntary) advanced bonus lab assignments (checked by TAs). All partial results (including the lab and the exam) are only valid during this academic year. Your end grade is the total points scored divided by 1000. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Permitted Materials during Tests	Pen/pencil. Everything else is not allowed: no books, notes, calculator, etc.	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Ir. O.W. Visser	
Co-Instructor	T.V. Aerts	

CSE1405	Computer Networks	5
Responsible Instructor	Prof.dr.ir. F.A. Kuipers	
Contact Hours / Week x/x/x/x	0/0/0/4 lecture; 0/0/0/4 shared lab	
Education Period	4	
Start Education	4	
Exam Period	4 5	
Course Language	English	
Course Contents	Computer networks form the basis of all digital communication. They are typically structured in layers. Each layer has clearly defined functions and algorithms, which we will discuss in class. Specifically, the course deals with the following layers: - Data link layer - MAC sublayer - Network layer - Transport - Application For each of these layers, the course introduces security issues and defenses. Furthermore, it provides an introduction to cryptographic notions utilized by the defenses.	
Study Goals	1) The student can describe the principles of layered architectures, in particular the functions and purposes of all layers for the OSI reference model 2) The student can describe the core algorithms of each layer and provide arguments why these algorithms fulfill the function of the respective layer 3) The student can apply the core algorithms to toy examples 4) The student can design variants of core algorithms for a given scenario. 5) The student can identify security issues of each layer and choose a suitable type of solution 6) The student can exemplify the concepts of ethical hacking and responsible disclosure	
Education Method	Lectures (8 weeks x 2 lectures x 2 hours) during these, new concepts are illustrated through the discussion of multiple examples from the book. Lab (5 weeks x 4 hours) during these the student gains experience with commonly used Internet software (e.g. Wireshark). Additionally the student applies given abstractions and network concepts.	
Literature and Study Materials	Computer Networks, Andrew S. Tanenbaum, 5th or 6th edition. International English edition. Lecture Notes for Network Security	
Assessment	Written exam on paper, closed-book. There is a single resit opportunity of the exam. Exams are a combination of multiple choice questions and open (computation-based) questions. The lab work is not graded. The final tests is your final grade. MapleTA or weblab is used, if the size of the student population allows it. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Permitted Materials during Tests	Calculator	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Ir. O.W. Visser	

CSE1500	Web and Database Technology	5
Responsible Instructor	Dr. C. Lofi	
Responsible Instructor	U.K. Gadiraju	
Instructor	Dr. A. Katsifodimos	
Instructor	C. Koutras	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/4/0/0 shared lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Expected prior knowledge	Knowledge of basic algebra. Proficiency in at least one programming language.	
Course Contents	Web & Database Technologies is a first-year course that provides an introduction to web and database technologies and programming. Topics related to web technology and programming include: = Introduction to the web and HTTP; = Introduction to web development and app design; = Front-end development: HTML, CSS, JavaScript; = Back-end development: Node.js. Topics related to database technologies include: = Introduction to databases; = Overview of database Languages and architectures; = The basic relational model; database design theory; = SQL: data definition, constraints, updates, queries, views, triggers; = Relational databases and object persistence; = Introduction to non-relational database systems: key-value stores, document stores, graph databases.	
Study Goals	Upon completion of this course, participants will have gained knowledge of web and database system concepts and the ability to: 1. Explain the basic architecture of the Internet and the web; 2. Analyse the requirements for a web application given a description of an application idea; 3. Create an application front-end with HTML and CSS based on a given static design; 4. Create an interactive web application with client-side JavaScript; 5. Develop server-side code for authorization, cookies, sessions and database access; 6. Compare the strengths and weaknesses of different data management techniques, describing their most suited applications through meaningful examples; 7. Enumerate and define the main elements of the relational model; 8. Describe and design SQL queries for the creation, altering, and manipulation of tables, indexes, and views; 9. Develop logical database schema, with principled design that enforce data integrity; 10. Prototype and deploy database applications using open-source database systems.	
Education Method	Lectures, and practical work (lab assignments).	
Literature and Study Materials	In addition to the official course book(s), students will be provided with: = The slides presented in class, cross-referenced to the content of the book(s); = Additional study materials, selected to provide insights on the topics covered during the lectures.	
Practical Guide	All necessary information about the practical setup (schedule, grading, exams, resits, etc.) of the course is available in the course syllabus (accessible via Brightspace).	
Books	WEB There is no required book for the web technologies part of the course. DATABASES The book is not required but helpful: Fundamentals of Database Systems, 7th Edition, Augustus 2016 (Global Edition). Authors: Ramez Elmasri, Shamkant Navathe. ISBN10: 1292097612. ISBN13: 9781292097619 URL: http://catalogue.pearsoned.co.uk/educator/product/Fundamentals-of-Database-Systems-Global-Edition/9781292097619.page	
Assessment	There are two exams: Midterm in W2.5 and Final in W2.10. Both exams are in written form, with multiple choice, short answer questions, and open questions. The final grade is calculated as Midterm exam 50% plus Final exam 50%. The two exams have individual Resits, typically conducted on the same day. One exam will cover all database topics, one exam will cover all web topics. To pass the course, students must obtain a grade greater or equal to 5 in both exams (midterm and final) and obtain an average of 5.8 or higher. There are mandatory lab assignments. The assignments are graded as either fail or pass. The entire course can only be passed if at least 50% of the lab assignments have been passed. Note that if at least 25% of the lab assignments have been passed, a repair option is provided. As per the Teaching and Examination Regulations (TER) Article 17A: the maximum obtainable result for the repair option is a 6.0 the repair opportunity takes place no later than the quarter following the regular examination opportunity in the same academic year	
Permitted Materials during Tests	No books or other materials are allowed during exams.	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Tags	Databases Software	

CSE1505	Information and Data Management	5
Responsible Instructor	Dr. C. Lofi	
Responsible Instructor	Dr. A. Katsifodimos	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Expected prior knowledge	Basic understanding relational query languages (SQL) and databases (as obtained in the Web and Database Technology Y1 Q2 course).	
Course Contents	This class also relies heavily on Algorithms and Data Structures. In this course you will learn how to model, manage, and query information in a Database Management System (DBMS). We will focus on different data modelling techniques, but also on aspects of query processing like for example query parsing or optimization. Also transaction management is a central topic of interest. The core focus is on classic relational OLTP databases, but we will also briefly touch alternative systems like the current NewSQL-trend and systems optimized for in-memory or OLAP applications. Additionally, NoSQL systems and their respective data models will be discussed. - Database normalization: modelling in an unambiguous and non-redundant manner in relational tables such that inconsistencies and anomalies are avoided. - File-organization and indexing: the data structures and their associated algorithms to store data in memory and on disk, such that it can be retrieved and processed efficiently when needed. - Query processing and optimization: the techniques that are used by a DBMS to execute certain SQL queries as efficiently as possible, taking into account how the data is stored and which indexes are present. This also covers Relational Algebra. - Transaction management and concurrency control: the techniques that a DBMS offers to allow different users and application to operate in parallel on the same data without interfering with each other or causing inconsistencies in the data. - NewSQL and alternative system: the systems beyond traditional relational database design space optimized for specialized requirements like extreme OLTP performance, OLAP queries, or in-memory storage. - NoSQL and non-relational data: in the recent years, systems not relying on the relational model became popular. We will also discuss how this affects data modelling, especially with respect to document models, key-value models, wide-columnar models, but also graph models.	
Study Goals	The course aims at teaching the basic competencies required to design and understand applications interacting with relational database management systems (RDBMS). Students will gain an insight into how RDBMS are implemented, learn about the available design choices, and the trade-offs imposed by different designs. This allows students to competently identify and solve problems in application design, and to improve and tune data models and queries. We will also deal with non-relational systems and discuss how they affect the data processing workflow. - Perform Data Modeling and be able to: -- Design and implement normalized relational schemas in a database system, by designing Entity Relationship diagrams and translating them to relational tables. -- Describe alternative data models such as the document model, key-value model, or graph models and analyze the trade-offs as in comparison to the relational model. -- Analyze the requirements of a given application scenario and be able to derive a suitable data model. - Get a grip of the internals of relational database systems by being able to: -- Describe and analyze the two main ways of storing and representing data in relational databases as well as analyze their performance trade-offs with respect to input/output (IO) costs. -- Describe different index structures and analyze their impact on query response time for range-, point- and join queries. -- Analyze the trade-offs between different physical operators for database operations such as joins, aggregates and filters. -- Describe and apply the query optimization process and design physical execution plan and compute the costs for their relational operations. -- Explain the need for transaction protocols and illustrate database update anomalies that can arise in the absence of such protocols. Be able to apply such protocols for given scenarios. -- Evaluate and apply different recovery algorithms such as UNDO/REDO logging, under different types of failures and their frequency.	
Education Method	Lectures and Homework Lab	
Computer Use	Students will require access to a personal computer. The labs will deal with larger data sets, so some HDD space (several GB) will be needed.	
Literature and Study Materials	The course and evaluation is based on the provided lecture slides. For alternative explanation or more in-depth studies, the recommended book can be used.	
Books	Fundamentals of Database Systems by Ramez Elmasri and Shamkant Navathe, Pearson. Preferred 7th edition, but also the older ones are usable. Database Systems: The Complete Book by Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom (2nd edition preferred, older ones also OK). These books are NOT required, but might be helpful for more in-depth studies. Additional (optional) books/resources are mentioned in the lecture.	
Assessment	There are two exams: Midterm and Final. The final grade is calculated as Midterm exam 50% plus Final exam 50%. The two exams have individual Resits, typically conducted on the same day. To pass the course, students must obtain a grade greater or equal to 5 in both exams (midterm and final) and obtain an average of 5.8 or higher. There are mandatory lab assignments. The assignments are graded as either fail or pass. The entire course can only be passed if at least 50% of the lab assignments have been passed. Note that if at least 25% of the lab assignments have been passed, a repair option is provided for the labs.	
Permitted Materials during Tests	1 handwritten sheet of A4 paper (any content chosen by the student, writing front and back permitted).	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	

Co-Instructor	Dr. A. Katsifodimos
Co-Instructor	C. Koutras

Year	2023/2024
Organization	Electrical Engineering, Mathematics and Computer Science
Education	Bachelor Computer Science and Engineering

Bachelor 2nd year TI 2023

Prerequisites	Please note! The entry requirements are mentioned in the regulations https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/regulations/. If there are differences between what is mentioned here and what is mentioned in the regulations, that what is mentioned in the regulations is binding. In the second year the following course has entry requirements: You may only participate in CSE2000 Software Project if you have passed the courses CSE1100 OOP, CSE1305 Algorithms and Datastructures, CSE1105 OOP Project, CSE1110 Software Quality and Testing and CSE2115 Software Engineering Methods. If you didnt pass the required courses/projects and think that not participating in the project will result in unduly study delay you may hand in an application for an entry requirements exemption. You will receive more information during assemblies, via Brightspace and by e-mail.
Introduction 1	Registration for exams and pre-registration for projects For the CSE2000 Software Project you pre-register during Q2. You will be informed about this by e-mail and during the General Assemblies. For the exams of all the courses you have to register at least two weeks before the day that they take place. More information is available on: https://www.tudelft.nl/en/student/education/examinations/registration-for-examinations/. If you decide after your register that you dont want to participate in the project or exam dont forget to cancel your registration.
Introduction 2	Variants The first three quarters of the second year you follow a Variant. There are three Variants: Multimedia, with the courses Signal processing, Image processing and Multimedia Analysis; Systems, with the courses Digital Systems, Embedded software and Operating Systems; Data, with the courses Big Data Processing, Data mining and Computational Intelligence. To finish your Bachelor CSE you must have completed one of the Variants as a whole.
Introduction 3	Preview year 3: In the first semester of the third year you do your minor, which you choose this year. For more information, go to: http://studenten.tudelft.nl/en/students/education/minors/. Would you like to go abroad in your third year? Start your orientation now on https://www.tudelft.nl/en/eemcs/study/exchange.

CSE2000	Software Project	15
Responsible Instructor	Ir. O.W. Visser	
Responsible Instructor	H. Wang	
Responsible Instructor	Ir. T.A.R. Overklift Vaupel Klein	
Contact Hours / Week x/x/x/x	0/0/0/4	
Education Period	4	
Start Education	4	
Exam Period	4	
Course Language	English	
Course Contents	The software project is carried out in a group of 5 students, who work together as a software development team. During the project, the team produces a product that is commissioned by a client and solves a real-world problem. Over the course of the project, the student team experiences the entire trajectory of a real-world software development project, including: - problem analysis, feasibility study, research, and risk analysis, - requirements engineering, - specifications and quality requirements, - implementation, - testing and validation, - delivery of a working product, including necessary documentation. The team tackles this challenge by integrating their knowledge and experience from previous courses and projects in the curriculum.	
Study Goals	At the end of the software project, students will have/demonstrate tangible proof that: LO1: They can carry out an entire software development cycle with success, from problem definition, researching solutions through testing the product. LO2: They can work in a team of developers addressing a real-world problem, in collaboration with a coach and a client. LO3: They can acquire the relevant information and knowledge from literature and different stakeholders. LO4: They can effectively choose a development strategy and execute a development process according to that strategy. LO5: They can establish the necessary quality requirements for a product and carry out the tests necessary to determine that the product fulfils those requirements. LO6: They can present and report a complete and convincing explanation of the development process and the product results. LO7: They can take societal, economical, ethical and legal aspects into account in the development of their solutions.	
Education Method	Project education with weekly feedback moments; problem-based learning. Sessions about technical writing, teamwork and responsible computer science.	
Prerequisites	Students are only allowed to get access to CSE2000 Software Project if the following courses are successfully completed: - CSE1100 Object Oriented Programming - CSE1105 OOP Project - CSE1110 Software Quality and Testing - CSE1305 Algorithms & Datastructures - CSE2115 Software Engineering Methods.	
Assessment	Assessment happens at the end of the project based on the final report, the code and test quality, and the final presentation. The committee will determine the grade based on: 1. the research phase: problem analysis, research/design methodology, and requirements engineering. 2. the way the project was executed; this includes planning, how the group communicated with all involved people, and the dedication of the students. 3. the quality of the final product, quality assurance thereof, and the accompanying documentation as well as the difficulty of the assignment in correlation with the achieved results. 4. the final report, including ethical evaluation of the product and/or its domain. 5. the final presentation and Q&A. The final grade for each of the project dimensions mentioned above should be at least a 5.0, and the final combined grade should be at least a 5.8 in order to pass this course. In exceptional cases the committee may decide that there is need for an individual adjustment of the group grade. For instance, if an individual within the group did not contribute sufficiently, the grade of that individual may be lowered. For the final report, a repair opportunity (resubmission) for a maximum grade of 6.0 is offered in case of a failing grade. Due to the nature of the project (group work for an (external) client for a fixed period of time) there is no repair opportunity for the other parts of the project.	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) ATTENTION: Participation in this course requires a mandatory registration via ProjectForum. The course registration will be open in weeks 2.8 & 2.9 of the academic year. After this deadline you can no longer enrol for the course. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/ The Software Project is a fulltime course (15 EC - 42 hours per week). Therefore we expect you to fully focus on your participation in the project when you enrol. You are expected to not take any other courses during the same quarter in addition to the software project. Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE).	
Special Information	This course makes use of a course email: sp-cs-ewi@tudelft.nl For matters related to enrolment approval / entry requirements, please contact: programmecoordinator-EEMCS@tudelft.nl	
Co-Instructor	Dr. M. Skrodzki	

CSE2115	Software Engineering Methods	5
Responsible Instructor	Dr. A. Panichella	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/4/0/0 lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Course Contents	The course covers the most important software engineering practices needed to build high-quality software systems. The overall goal is to give the students the instruments to become Software Professionals. The course covers both high-level software project management, and low-level software design techniques needed to create evolvable and maintainable systems. Specific topics covered include design patterns, requirements engineering, UML modeling, software architecture, and software analytics. In the lab work, students will apply a selection of techniques to a group project they will develop and to assignments on theoretical aspects.	
Study Goals	(1) Students can select appropriate software engineering methods, which are designed and used to build maintainable and evolvable software systems. (2) Students can judge the benefits/drawbacks of the presented software engineering methods that are commonly used in current software development. (3) Students can use presented software visualization techniques and software analytics to obtain an overview of an existing software system and to detect anomalies in the structure of the code. (4) Students can judge the real benefits/drawbacks of using certain design patterns and software validation techniques in a given software project. (5) Students can derive the valid requirements for a new software project, given natural-language descriptions and interaction with stakeholders. (6) Students can apply the most appropriate software engineering practices in a project, given the context of development.	
Education Method	Labwork + Lectures	
Literature and Study Materials	Materials will be provided via Brightspace	
Assessment	----- Final grade composition ----- The final grade of this course is calculated using the following weights; * Written exam: 50% * Lab work: 50% * Oral Presentations: Pass/Fail * Attendance to weekly meetings (Oral Presentations): Pass/Fail ----- General remarks ----- * Each grade must be sufficient to pass the course. * Passing the course is conditioned by passing the Oral Presentation module. * It's mandatory to contribute to all assignments; there must be visible activity in the code repository (the contributions should be of reasonable size, to be judged by the lecturers). Not contributing to one of the parts of this course can result in a failing grade for the course. Students are responsible for checking that they've contributed to all assignments. * The labwork is graded with the same grade for the whole group; in case of unequal work distribution, etc. lecturers may decide to modify grades of individual students such that the grade represents the work delivered by that student. * Partial grades are not carried over to the next edition of this course. * In case you have completed Oral Presentations before then you may be eligible for an exemption for the attendance to the weekly lectures. Check Brightspace for more information. ----- Resits ----- * The re-sit will replace the grade for the exam only. * Lab work cannot be recovered through the re-sit but must be completed sufficiently during the course. * For the Oral Presentations part we offer a single resit possibility. ----- Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Special Information	Contact with the course staff via: sem-cs-ewi@tudelft.nl (use your TU Delft email address).	
Co-Instructor	F. Mulder	

CSE2120	Concepts of Programming Languages	5
Responsible Instructor	C.B. Poulsen	
Responsible Instructor	T.V. Aerts	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 lab; 0/0/2/0 tutorial	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Expected prior knowledge	Required: - No courses are formal entry requirements to this course.	
Course Contents	Strongly advised: - CSE1100 Object Oriented Programming: Knowledge about object-oriented programming in languages like Java. - CSE1110 Software Quality and Testing: Knowledge about how to test your code and how to write understandable and maintainable code. The design of a programming language is the result of a selection from a general collection of programming language concepts. With some frequency, new languages are introduced that combine concepts in a novel manner and sometimes introduce new concepts. During their careers, computer scientists will have to work with more than one generation of programming language. In order to learn new programming languages, a computer scientist should understand the basic principles in the design of programming languages. In this course we study these principles. The topics covered in the course include: * Syntax and interpretation: programs as data, abstract vs. concrete syntax, interpretation * Functional programming: higher-order functions, lexical scope, function closures, currying and partial application, functions over data (especially map, fold, and filter). * Mutation and memory: implementing loops and recursion, tail calls, heap memory allocation and reuse, concurrency and parallel execution. * Lazy evaluation and infinite streams: lazy vs. eager evaluation, thunks, streams, memoization. * Types: a type as a set of values together with a set of operations, primitive types, compound types, type safety and errors caused by inconsistent use of values, association of types to names, type checking, type inference. * Objects and encapsulation: privacy and visibility of class members, interfaces revealing only method signatures, inheritance, subtyping.	
Study Goals	By the end of this course, you will be able to: 1. Use algebraic data types and pattern matching to implement a definitional interpreter for a programming language concept. 2. Use a modern high-level programming language such as Scala to implement, test, and debug definitional interpreters and type checkers for languages with simple types. 3. Define example and counter-example programs that use a programming language concept correctly and incorrectly; for example, variables and lexical scope in a program using function closures. 4. For multiple programming languages, identify program properties checked statically and program properties checked dynamically. 5. Discuss how programming concepts interact with the definition and implementation of other language features. 6. Describe trade-offs of the functional approach (defining a function for each operation with the function body providing a case for each data variant) and the object-oriented approach (defining a class for each data variant with the class definition providing a method for each operation).	
Education Method	The course consists of: - lectures - self study of the material - making of assignments in WebLab During lab hours, teaching assistants will be available for helping students. Students are expected to bring to the lecture pen and paper for making notes and solving assignments.	
Books	We use multiple online, free-to-use books: - Programming Languages: Application and Interpretation (2nd edition), S. Krishnamurthi, free online book available at http://cs.brown.edu/courses/cs173/2012/book/ - Object-Oriented Programming Languages: Application and Interpretation, E. Tanter, free online book available at https://users.dcc.uchile.cl/~etanter/ooplai/ Additional reading material will be provided through Brightspace or WebLab by the lecturers of the course.	
Assessment	The final grade of the course consists of the following components: - Mandatory Lab (pass/fail) - Computer Exam (100%) - (optional) Lab Bonus (max 0.5 points) Final grade calculation: - Mandatory Lab failed -> 1 - Mandatory Lab passed and Exam Grade < 5.8 -> Exam Grade - Mandatory Lab passed and Exam Grade >= 5.8 -> Exam Grade + Lab Bonus Bonus: A bonus of up to a maximum of 0.5 points on the final grade may be obtainable through extra assignments in the lab. The bonus only applies if the exam (or resit) grade by itself is a passing grade (>= 5.8), i.e. the bonus cannot bring a grade to a passing grade. Repair possibilities:	

CSE2215	Computer Graphics	5
Responsible Instructor	Prof.dr. E. Eisemann	
Responsible Instructor	Dr. R. Marroquim	
Project Coordinator	M.L. Molenaar	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 4/0/0/0 lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Expected prior knowledge	To avoid excluding students from participation that caught up with the necessary knowledge over the summer, the following courses are not a prerequisite. Nevertheless, knowledge in these areas is required.	
Course Contents	CSE1100/TI1206 Object-oriented programming CSE1305/TI1316 Algorithms and Data Structures CSE1200/TI1106M Calculus CSE1205/TI1206M Linear Algebra This course introduces computer graphics and its participants will learn about the scientific challenges and problems of image synthesis and gain working knowledge in this area. The focus will lie on rendering (the process of producing images on a screen). The course will present the foundations of image synthesis, as well as the modelling of shape and appearance of objects. The introduced elements are often general and apply to a wide variety of topics even outside of the area of Computer Graphics. Examples include geometric representations, hierarchical structures, or projective geometry, which will also reoccur later in the curriculum in various courses. A good basis in linear algebra is important for this course. Computer Graphics applies fundamental aspects of Computer Science (parallel programming, data structures, algorithms...). The course will give an overview of the field, but will create links to contemporary results and research, which will offer the possibility to further deepen certain knowledge and to gain an outlook on recent trends in research. A preliminary outline is as follows: 1) Introduction, Objects and Images 2) An introduction to C++ 3) Transformations and Homogeneous Coordinates 4) Projective geometry 5) An introduction to OpenGL 6) Materials and shading 7) Shadows 8) Whitted-Style Ray Tracing 9) Acceleration Structures	
Study Goals	This course enables the student to: -S1- Explain and compare the structure and properties of standard algorithms and data structures linked to Computer Graphics. -S2- Execute and visualize standard algorithms and data structures. -S3- Use mathematical methods to analyze, create, apply algorithms and data structures, as well as understanding time and space complexity of image-generation algorithms -S4- Apply mathematical modeling and theory of geometric computations and transformations, object representations, simulation, and encoding. -S5- Implement algorithms and data structures using the C++ programming language and OpenGL. -S6- Apply the knowledge obtained in this course to problems of other fields During this course, the participants will receive an overview of computer graphics and gain familiarity with the field. They should be able to know the basic terms related to this domain, understand the representation of images, objects, and the image formation process, including the theory, optimizations, and underlying physical processes. Hereby, the notion of physical models will be explored with examples like material representations. Further, they will learn about the difficulties and challenges related to advanced image synthesis. Via practical assignments and lab work, the participants will become familiar with 3D APIs and be able to reproduce parts of the course in practice. They will learn that graphics can be useful in various contexts to visualize intermediate results and data. They further learn to create their own applications. It will become evident that graphics can even serve as a means to solve problems studied in other disciplines. Given the close link to linear algebra, students will extend their knowledge and learn to calculate with and make use of advanced concepts, namely homogeneous coordinates.	
Education Method	The course will consist of about 10 lectures (2 hours each) and 10x4h guided practical sessions. The latter will also serve as a basis for the course project. Initial sessions will be dedicated to background knowledge and recaps and will cover the basics of the programming language used in the course (C++).	
Literature and Study Materials	The following book is highly recommended for this course: Fundamentals of Computer Graphics, 4th Edition, Steve Marschner and Peter Shirley, CRC Press (e-book available from the TU Delft Library)	

Assessment

The following books can be used as extra resources:

Computer Graphics. Principles and Practice by James D. Foley, Andries VanDam, Steven K. Feiner - Addison Wesley
- Very complete, goes far beyond the course but covers all topics

Mathematics for 3D Game Programming & Computer Graphics by Eric Lengyel - Charles River Media (contains also exercises)
- More focus on linear algebra, contains not all topics of the course but offers exercises

Real-time Rendering by Tomas Akenine-Möller, Eric Haines, Naty Hoffman - Peters, Wellesley
- Complete but higher level, with many more content related to real-time approaches

Real-Time Shadows by Elmar Eisemann, Michael Schwarz, Ulf Assarsson, Michael Wimmer - Taylor & Francis
- only suggested for learning more about the particular topic of shadows

The final grade consists of two partial grades: Exam grade [E] and a Practical Grade [P]

The final grade is computed via $\text{ROUND}(0.5 \cdot E + 0.5 \cdot P)$, where ROUND is the mathematical rounding in .5 steps.

Following the TU Delft regulations, to pass the course each partial grade E and P should be ≥ 5.0 , and the final grade should be ≥ 6.0 .

[E] Exam grade:

Physical exam: may contain multiple-choice and/or programming questions and/or open-ended questions. A simple calculator without programming capabilities is permitted. No course or other materials are allowed.

[P] Practical grade:

The Practical grade consists of two parts: Assignment score [A] and Final Project score [F] (both rounded on .1)

The Practical grade is given as: $P = 0.4 \cdot A + 0.6 \cdot F$ and mathematically rounded in .1 steps.

The two parts of the Practical grade [P] are defined as follows:

[A] Assignments:

There are 6 individual assignments, of which 5 will be evaluated with the weights: 15%, 25%, 25%, 25%, 10%
Each assignment should be completed until ~1 week after the handout (actual deadlines will be available via Brightspace).
Please note that committing fraud in one of the assignments leads to the invalidation of the whole Assignment grade [A] and eliminates the option for any repairs.

[F] Final Project:

The final project score is achieved in groups but students can receive individual scores based on their performance.
The group distribution is determined by the scores of the first three individual assignments.
Please note that committing fraud in the final project leads to the invalidation of the project score [F] and eliminates the option for any repairs.

Resits:

There will be a resit for the Exam - participation is possible without any conditions, and the maximum grade of the first exam and the resit will constitute the exam grade E.

Repairs:

There will be a repair opportunity to replace the Assignment score in the form of a multi-component assignment. This repair assignment can involve code creation/hand-in in the context of a computer assessment. The conditions for participation reflect those in TER Art 5.5, with the assumption that the Assignment score is considered a partial grade.

There is a repair for the Final Project score linked to certain conditions;

It will take the form of a new individual project and is only available to those that have an average ≥ 6 points in their Assignment score [A] and a ≥ 6 for their first exam grade (not the resit). It only allows the participants to increase their Final Project score [F] to maximally a 6.

The use of any repair opportunity results in the Practical grade [P] being bound to maximally a 6.0.

Enrolment / Application

Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)

Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE).
Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: <https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/>

Co-Instructor

Dr. M. Skrodzki

CSE2220	Signal Processing	5
Responsible Instructor	Prof.dr.ir. R.L. Lagendijk	
Responsible Instructor	E. Isufi	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 4/0/0/0 lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Required for	Image Processing. Multimedia analysis. Highly recommended for Data mining and Human-computer interaction. Also highly recommended for Computer Science M.Sc. track Data Science & Technology, and the Cybersecurity specialisation	
Expected prior knowledge	Required: - Linear algebra: vector and matrix manipulations. Strongly advised: - Calculus: series, integration, complex numbers, complex exponents, Euler's formula. - Special attention is asked for complex numbers. Self-check exercises and study material is provided via Brightspace	
Course Contents	Digital signal processing is used in many modern computer sciences systems and applications. Examples are machine learning, artificial intelligence, and big data analysis; content recommenders in information systems; speech, music and image content based retrieval and searching; music and video compression; sensor data processing in embedded systems; bioinformatics and medical data analysis. This course deals with the foundations and principles of digital signal processing. The first part concentrates on acquiring digital signals (sampling) and the basic linear filtering operations and convolution. The second part of the course introduces the concept of frequency or Fourier description of signals and systems. This concept is the foundation of many of today's computer science-based systems and applications, and has found wide applications in the processing of a variety of sound, music, sensor, image, video and other multimedia information	
Study Goals	1. Properties of digital signals and systems. The student is able to explain, compute and apply the properties of time-discrete linear time-invariant systems and the input-output relation; explain and use the principle of superposition; write arbitrary signals in terms of the sum of impulse signals. 2. Linear time-invariant (LTI) filters and convolution. The student is able to explain, determine and apply the impulse response of a time-discrete LTI system such as FIR or IIR filter; explain the relation between impulse response, input-output relation and convolution; manually (for short signals) or numerically (using Python, for long signals) convolve the input signal and the impulse response to compute the output signal; explain and apply the properties of convolution, including the difference between linear and circular convolution; recursively compute the output of an IIR filter; implement and visualize convolution via Jupyter Notebook; implement and apply FIR filters in practical exercises and assignment, and visualize and interpret the results. 3. Sampling. The student is able to explain and apply the process of data acquisition (sampling) and Shannon's sampling theorem; explain the concepts of oversampling, undersampling and aliasing; explain how to avoid aliasing; use of zero-order or linear interpolation for the reconstruction of time-continuous signals; make audible (using Jupyter Notebook) original and processed audio signals and interpret the results. 4. Discrete Fourier Transform (DFT). The student is able to write a periodic signal as the sum of complex exponentials; explain the concepts and differences of Fourier series, DTFT and DFT; explain and interpret the complex spectrum, magnitude spectrum and phase spectrum ; compute and visualize manually (for short signals) or numerically (for longer signals) the magnitude and phase of DTFT and DFT coefficients, and interpret the results; explain the difference between the numerical results computed using the DFT for finite-length signals and the theoretical results computed using the DTFT for infinite-length signals; explain the principles of the fast Fourier transform (FFT); apply the DTFT and DFT to practical exercises and assignments. 5. Frequency response of LTI filters. The student is able to compute manually (for short impulse responses) or numerically (for longer impulse responses) the frequency response of an LTI system using DTFT and DFT; visualize and explain the magnitude and phase response of an LTI system; evaluate convolution via product of frequency response of the filter and Fourier transform of the input signal; compute the frequency response of cascaded filters; numerically determine the impulse response of an FIR filter with desired frequency response using the inverse-DFT method. 6. Applications of filters and DFT. The student is able to apply theory and practice to (simplified) versions of relevant computer science-based signal processing problems and assignments, such as music compression and recognition.	
Education Method	The course consists of - 7 plenary lectures in which the main concepts are explained and discussed; - 7 working group in which theory and practical (based on Jupyter Notebook) exercises are solved and discussed; - 6 hands-on sessions in which practical (based on Jupyter Notebook) assignments need to be solved and approved by the instructor.	
Computer Use	The course includes a compulsory set of computer (based on Jupyter Notebook) assignments that need to be solved and approved by the instructor during the hands-on sessions. Jupyter Notebook software is pre-installed on the computers available during the hands-on sessions. Students can also install Jupyter Notebook software on their own computer.	
Literature and Study Materials	The course loosely follows the book 'Signal Processing First' James H. McClellan, Ronald W. Schafer and Mark A. Yoder ISBN 0-13-120265-0 Prentice Hall. Alternatively, also on-line material may be used, such as 'The Scientist and Engineering's Guide to Digital Signal Processing', S.W. Smith (http://www.dspguide.com/pdfbook.htm). Slides used during the lectures are made available via Brightspace. Theory and practical exercises, as well as the Python assignments (via Jupyter Notebook) are available for download via Brightspace. During the course several pointers to on-line MOOCs and video lectures will be given	
Practical Guide	Study material and the hands-on assignments can be downloaded via Brightspace.	
Assessment	The first partial examination is in week 5, and deals with the materials covered in plenary lectures, working groups and Python assignments in week 1-4. The result of this partial exam counts for 30%. The second partial examination is scheduled at the end of the quarter, and covers all course material with emphasis on the materials covered in plenary lectures, working groups and Python assignments in the second half of the course. The result of this partial exam counts for 70%. A resit exam, covering all materials of the plenary lectures, working groups and Python assignments, is scheduled at the end of the second quarter. The Python hands-on assignments are integral part of the course and must be submitted for approval within the weekly-indicated deadlines. Students can solve their assignments at home (using an own computer) or during the lab sessions. Completion of the hands-on assignments are a prerequisite for entering the second partial exam or the resit exam. Students can re-submit at most one failed Python hands-on assignment during the course to still be eligible for the second partial exam. This must be done	

	within the sessions of the spare labs in weeks six and ten. Completion of the hands-on assignments are a prerequisite for entering the second partial exam or the resit exam. The final grade at the end of the quarter in which the course is lectured is determined as follows. 1. The grade (rounded to one decimal) of the first partial examination counts for 30%. 2. The grade (rounded to one decimal) of the second partial examination counts for 70%. 3. The Python hands-on assignments have to be completed and approved by the instructor. The approval of each assignment must be done within the week set for that assignment. In case of unforeseen circumstances (like illness) the instructor can allow for a delay of maximally one week, and only if such approval is obtained at the latest in the week the assignment should have been completed. Validity of partial examination results and completed Python (Jupyter Notebook) hands-on assignments. * The results of the partial examinations expire at the end of the quarter in which the course was lectured if the Python hands-on assignments have not been completed and approved by the instructor before the second partial examination. * The completed Python hands-on assignments are valid for an entire academic (study) year, but expire at the end of the academic (study) year. The grade of the resit examination is determined as follows. 1. The grade of the resit examination. 2. The Python hands-on assignments completed before the resit examination. Students who had not yet completed the hands-on assignments during the regular sessions will be given the opportunity to do so during the two spare labs to verify and approve all (remaining) assignments. These appointments take place in the weeks before midterm and the endterm. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)
Permitted Materials during Tests	The lecturers have prepared one page with key equations (formuleblad). This page is permitted during the examination. It is also allowed to make notes on this one printed page.
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/
Maximum number of participants	120
Self test	Fourier transforms, DTFT and DFT rely heavily on the description of signals as a sum of complex exponentials. It is important that the student brings operational knowledge of complex numbers (see first years Calculus). A set of self-tests is available as well as self-study materials. Complex exponents are assumed background knowledge and not discussed during the lectures.
Co-Instructor	A.W.F.A.M. Elnouty

CSE2225	Image Processing	5
Responsible Instructor	Dr. X. Zhang	
Responsible Instructor	A.W.F.A.M. Elnouty	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/4/0/0 lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Expected prior knowledge	Linear Algebra, Signal processing, Machine Learning	
Course Contents	In this course we will extend the theory of 1D signal processing to 2D digital signals: images. We will discuss image properties such as pixels, resolution, histograms, 2D spatial-invariant systems (2D-convolution filters), image enhancement, color spaces & filtering, morphology, 2D Fourier transformations, Hough transform and patch descriptors. This course will also provide students with the experience of implementing a real-life image processing system and dealing with the challenges entailed by researching different approaches and finding a creative solution for these issues.	
Study Goals	At the end of the course you will be able to: - Explain the image processing algorithmic steps and the practical application of standard image processing techniques such as: image filtering, morphological operations, edge detection, Hough Transform, patch description, etc. - Correctly implement the main steps of the algorithms discussed in each lectures using Python. - Design in Python a working and complete applied image processing system by integrating and adapting the the image processing techniques discussed in the lectures. - Assess each component in the designed image processing system by considering their accuracy and speed and failure cases.	
Education Method	The course uses: - Interactive lectures: explaining the information and deriving some of the properties of the algorithms discussed together with the students. We do exercises during the lectures and ask questions. - Hands-on sessions for lab assignments: the students are given practical (based on Jupyter Notebook) individual lab assignments focused on algorithms or parts of the algorithms discussed in the lecture that need to be solved and approved by the instructor. - Project sessions for a python-based project: The students are required to solve a quarter-long project for a real life image processing problem. The students' progress for the project's design & implementation is monitored and guided to make sure students are on track. This project is solved in groups.	
Computer Use	The course includes a compulsory set of computer (based on Jupyter Notebook) assignments that need to be solved and approved by the instructor during the hands-on sessions. Jupyter Notebook software is pre-installed on the computers available during the hands-on sessions. Students can also install Jupyter Notebook software on their own computer. The course includes a compulsory project that needs to be implemented and evaluated by the instructor before the end of the quarter.	
Literature and Study Materials	The course loosely follows the book 'Digital Image Processing' Rafael C. Gonzalez Richard E. Woods. 3rd ed. Additionally Richard's Szeliski Book: https://szeliski.org/Book/ is used. Slides used during the lectures are made available via Brightspace. Theory and practical exercises, as well as the Python assignments (via Jupyter Notebook) are made available for download via Brightspace. During the course several pointers to on-line MOOCs and video lectures will be given.	
Assessment	- Final written exam (50% of course grade): a written exam Scheduled at the end of the quarter is offered covering & testing the acquired knowledge level of the topics discussed in the lectures. A grade above or equal 5.0 is required to pass the course & for this grade element to contribute to the total course grade. - Hands-on Python Lab assignments (25% of course grade): Pass/ Fail weekly practical lab assignments. They must be submitted for approval within the weekly-indicated deadlines. Students can solve their assignments at home (using an own computer) or during the lab sessions. A grade of above or equal 5.0 on the hands-on assignments (passing ~50% of the lab assignments) are a prerequisite for entering the final exam or the resit exam & for this grade element to contribute to the total course grade. - Project design & analysis (25% of course grade): A quarter-long python based project must be submitted for approval within the indicated deadlines. Students can solve their assignments at home (using an own computer) and during the project sessions. A grade above or equal 5.0 is required to pass the course & for this grade element to contribute to the total course grade. Resit: - Resit exam (50% of course grade): covering all materials of the plenary lectures, working groups and Python assignments, is scheduled at the end of the second quarter. - Repair Lab assignments (25% of course grade): students who have a grade between 4 and 5 can redo one of the lab assignments (choice by the lecturers). - Repair project (25% of course grade): an extra opportunity to submit the project is offered in Q3 with the limitation of a max grade of a 6.0 for passing the project requirements. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Dr. J.C. van Gemert	

CSE2230	Multimedia Analysis	5
Responsible Instructor	Dr. O.E. Scharenborg	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Course Contents	The course provides basic knowledge and hands on experience in content analysis for the full range of different types of multimedia, including audio, speech, text, images and video. The emphasis is on 1) techniques needed to develop systems that provide users with a variety of functionalities, e.g., search, organization, discovery and sharing; 2) designing and implementing experiments to support system development; academic thinking is a vital skill moving forward as a professional engineer. An important aspect of the course is learning to differentiate between building and evaluating applications from a technological and human point of view. The course builds on concepts from signal processing and on image processing (2-D filtering, visual features, segmentation and basic image classification). It introduces new concepts such as text classification, automatic speech recognition and multimodal video indexing. It makes the bridge between fundamental techniques and applications that allow users to interact with multimedia collections at a level that goes beyond a signal and encompasses aspects of human interpretation of the meaning and use of multimedia content.	
Study Goals	Study Goals Multimedia signals: Remembering: describe different sources of multimedia signals. Remembering: characterize the impact of editing on multimedia. Understanding: differentiate between objective and subjective descriptions of multimedia objects. Multimedia systems: Remembering: name the elements of the workflow of a multimedia system (in particular, a search engine). Remembering: describe the user goals that a multimedia system has been designed to meet and the domain in which it operates. Understanding/Analysing: establish a simple set of requirements with respect to which a multimedia system can be evaluated. Understanding: explain the use of basic machine learning techniques in the implementation of multimedia systems. Applying: carry out simple computations for evaluating multimedia systems. Reasoning: explain challenges of a specific example of multimedia content analysis. Evaluating: evaluate failure modes of multimedia descriptors or analysis methods. Audio Analysis: Remembering: describe and carry out the steps necessary to extract audio features used for audio analysis. Understanding: explain the advantages of basing audio features on human perception. Applying: build and test a simple audio classifier. Speech Analysis: Remembering: describe the basic elements of human language and its production. Remembering: name the modules of a speech recognition system and describe their function and interaction. Evaluating and Analysing: discuss characteristics of speech recognition transcripts and their impact on multimedia systems. Reasoning: reason about the effect of (mismatched) training and test data Image analysis: Remembering: name basic (global and local) visual features and describe their typical uses. Remembering: describe the steps necessary to extract basic (global and local) visual features and describe their typical uses. Remembering: describe, set up, and test a simple classifier for visual material. Applying: implement some basic image descriptors from its corresponding description. Analyse: compare and contrast different visual features with respect to a specific use case. Reasoning: hypothesise about how certain visual features may describe objects and depictions of objects in multimedia content. Video analysis: Remembering: name basic (global and local) visual features and describe their typical uses. Understanding: describe the differences between edited and unedited video Applying: implement some basic video descriptors from its corresponding mathematical description. Analysing: compare and contrast different video features or video analysis techniques. Reasoning: hypothesise about expected feature responses in edited video and experimentally validate this. Multi-modal analysis: Remembering: name and explain the difference between high- and low-level feature fusion. Evaluating: identify the weak points in various currently existing multimedia systems and offer constructive suggestions for improvements that are finally implemented. Creating: suggest a multimodal approach to an existing single modality solution. Creating and Evaluating: set up and evaluate a simple system for analyzing multimedia material. Critical thinking: Understanding: explain the difference in performance testing of an application from a technological and human point of view Evaluating: evaluating an application from a technological and human point of view Evaluating: setting up an experimental design to evaluate an application from a technological and human point of view	
Education Method	Lectures and labs	
Assessment	Labs (pass/fail) and a written exam. Participation is required at the labs. The written exam is a closed-book exam. A repair lab can be requested. A "pass" on the course labs cannot be carried over to future quarters. If the student passes only part of the course, all parts need to be retaken. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	J.A. Martinez Castaneda	
Co-Instructor	Dr. H.S. Hung	

CSE2310	Algorithm Design	5
Responsible Instructor	Prof.dr. M.M. de Weerd	
Responsible Instructor	S. Hugtenburg	
Responsible Instructor	Dr. E. Demirovi	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/4/0/0 lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Expected prior knowledge	Required: programming experience from CSE1100 or equivalent Strongly advised: experience with algorithms and data structures (priority queue, graph algorithms etc) such as from Algorithms and Datastructures (CSE1305); experience with constructing proofs, such as from Reasoning and Logic (CSE1300)	
Course Contents	Graph algorithms, greedy algorithms, divide and conquer, dynamic programming and network flow algorithms	
Study Goals	After completing this course, the student can apply, reproduce, and give relevant properties and definitions of 1. graph algorithms, 2. greedy algorithms 3. divide and conquer, 4. dynamic programming and 5. network flow algorithms. The student can also 6. prove the correctness of these algorithms, 7. determine the running time of algorithms both analytically as well as experimentally, 8. implement algorithms efficiently, and 9. provide well-argued advice about which problems can be solved best using which algorithms.	
Education Method	Lectures (possibly only online / pre-recorded) Each lecture includes a number of multiple choice questions and a selection of especially the difficult components of the material. Students are expected to study some of the topics at home. Tutorials (possibly only online) In these interactive sessions we take you through some example assignments (both implementation and written ones). Scheduled classroom lecture hours may be used for a combination of the above, as announced by the lecturer. Skill circuit assignments To train with the learning objectives, several assignments at exam level, but also more accessible ones are available on BrightSpace as elements in the skill circuit. For some of them feedback can be requested from teaching assistants or other students if handed in on time. Similarly, students can train for the digital test by making the implementation assignments in the skill circuit. Correctness of the code will be automatically checked.	
Literature and Study Materials	The study material consists of the book Algorithm Design by John Kleinberg and Eva Tardos (2005), Addison Wesley ISBN 13: 9780321372918 or 9780321295354 and some documents made available through BrightSpace (such as a manual of proving techniques and the master's method for solving simple recurrent relations).	
Assessment	The final grade of the course consists of the following components: Written exam part 1 (of 1.5h) in week 5 (W1) Written exam part 2 (of 1.5h) in week 10/11 (W2) Computer exam (3h) in week 10/11 (P) Final grade The final grade for the course is computed as follows: $W = 0.5 W1 + 0.5 W2$ If $P < 5$ or $W < 5$, the final grade is $\min(P, W)$. Else the final grade is $0.5 W + 0.5 P$ Both parts P and W can be retaken, in individual exam moments in the resit week. Written exam (W) Half-way and at the end of the quarter written examinations of 1.5 hours each are organized. The written exam primarily assesses proving skills and (application of) algorithms and algorithmic theory. Resitting W implies redoing both parts in a single three-hour exam. Computer exam (P) Algorithm design and implementation skills are assessed through a computer exam of three hours at the end of the course. The computer exam needs to be submitted through Weblab, but IntelliJ is available within the exam environment. These partial results are valid only in the current academic year. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Permitted Materials during Tests	None	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Special Information	course email address: ad-cs-ewi@tudelft.nl	
Co-Instructor	Dr. E. Demirovi	

CSE2315	Automata, Computability and Complexity	5
Responsible Instructor	Dr. M.T.J. Spaan	
Responsible Instructor	I.D. van Kreveld	
Responsible Instructor	S. Hugtenburg	
Responsible Instructor	Dr. J.W. Böhmer	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Expected prior knowledge	Proof techniques (such as by induction or contradiction), set theory, recursive definitions, trees and graphs. For example from CSE1300 or equivalent.	
Course Contents	Run-time complexity of algorithms. For example from CSE2310 or equivalent. This course introduces three areas of the theory of computation. First, automata are used to recognize words in (formal) languages, and we will discern several types of languages along with the types of automata that can recognize them. Second, once we have an intuition about languages, we discuss the topic of computability, where you will learn not only what kinds of problems a computer can solve but also how to prove this. Third, we will examine the class of computable problems and make a distinction between "easy" and "hard" problems and delve into the famous problem: $P = NP$? Study Goals After this course, the student can: 1. parse and construct automata and languages. 2. prove a problem to be computable. 3. manipulate and construct grammars for context-free languages. 4. describe the differences between different types of automata and grammars and their limitations, specifically that there are problems that are algorithmically unsolvable. 5. prove languages to be non-regular using the pumping lemma. 6. describe the classes P, NP, and NPC and explain why $P = NP$? is an open problem. 7. verify a proof that shows that a problem is NPC. Education Method Engaging interactive lectures for which the student studies the material at home. During the lecture, the students test their knowledge via multiple choice questions and the teacher explains difficult topics on demand. Lab course in the form of optional take-home assignments, 8 times during the quarter. Students solve these assignments individually and hand them in weekly according to predetermined deadlines. Students correct each others work followed by a round of feedback by teaching assistants. Literature and Study Materials 1. Textbook (see Books), chapters 0, 1, 2 (2.1-2.3), 3, 4, 5 (5.1), 7. 2. Supplementary material (lecture slides, video lectures). Books Michael Sipser, Introduction to the Theory of Computation, Third, International Edition, Cengage Learning. ISBN 978-1-133-18781-3. Assessment Note: if you do not use the international version, then section numbers and exercises will be different! A written, closed-book exam (see permitted materials during tests below). The final grade of the course is based on the final exam for 100%. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4). Permitted Materials during Tests You are allowed to bring a hand-written double sided A4 cheat sheet. No additional materials are permitted, including but not limited to: books, printouts of the slides, or devices. Enrolment / Application Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/ Co-Instructor Dr. M.T.J. Spaan	

CSE2420	Digital Systems	5
Responsible Instructor	Dr.ing. M. Shahraki Moghaddam	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 8/0/0/0 lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Expected prior knowledge	No prior knowledge is expected from the student.	
Course Contents	This course introduces the following aspects in digital design: combinational digital systems, sequential systems, complex combinational and sequential building blocks, reconfigurable hardware and complex digital systems. All the important aspects of digital design will be discussed: number systems, Boolean algebra, combinational logic, 2-level and multilevel networks and reduction techniques/design tools, timing hazards, Moore/Mealy machines, Finite State Machines (FSM) and more. The lab starts with the realization of simple circuits and will end with complex systems realized on an FPGA.	
Study Goals	The student (at the end of the course) will have learned the two fundamental notions of digital systems, namely combinational and sequential networks, and will have learned the basics of VHDL. First, the student will be able to translate non-digital functions to a digital representation. Second, the student is subsequently able to establish the needed combinational and/or sequential networks that have to obey specific design requirements (e.g., area, timing, and functionality). Furthermore, the student is able to model simple circuits in VHDL and understand them. The student will have gained enough knowledge to follow more advanced course, such as, computer architecture, computer systems, and embedded systems.	
Education Method	Lectures and lab.	
Assessment	* Multiple-choice exam and successful completion of lab. * Weekly online tests (5x) that can lead to "striking out an incorrect answer" at the first upcoming exam when 80% of the online tests questions are correctly answered. This rule does not apply for resits. There are two requirements for students to pass the course. First, an obligatory lab must be successfully completed. Only successful completions are registered. Second, a written exam (comprising 20 multiple-choice questions) must be taken resulting in a passing grade, i.e., 6 or higher. Resit & Repair Each year there are two possibilities (regular exam in the same quarter and a resit in the subsequent quarter) to take the exam according to the exam regulations that also defines the validity duration of the exam results. Students who do not finish the lab assignments in time are offered an extra lab to complete the assignments. In order to motivate students to study the course during the quarter it is given, an online test system (via Brightspace) is set up with up to 15 multiple-choice questions. The questions are put online in sets of 3 during a period of 5 weeks and relate to any of the material given in preceding weeks. Upon correctly answering 80% or more of these questions, 1 incorrectly answered multiple-choice of the first ensuing exam will be stricken. Once again, this online test result will only count for the first upcoming exam that is given in the same quarter and does not count for the resit nor for exams in subsequent years. Furthermore, interested and motivated students are given the opportunity to interact more with the lecturer via study groups in 5 meetings spread over 5 weeks. Registration is required in order to participate in these groups as the number of seats is limited. One task of students participating in these study group is to come up with multiple choice questions that can be used in the earlier mentioned online test. After discussion and vetting by the lecturer, each group will select 3 questions for the weekly sets. As a bonus for their participation, each student in each group will receive a pass for the three questions selected in the meeting. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Permitted Materials during Tests	Book and lecture slides, excluding old exams and tests.	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Maximum number of participants	120	
Co-Instructor	Ing. A.M.J. Slats	

CSE2425	Embedded Software	5
Responsible Instructor	Q. Wang	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/4/0/0 lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Course Contents	The course offers an introduction to real-time programming in an embedded system context. The student acquires the principles of real-time programming in C (interrupts, multitasking, event-driven application design, etc.) through a simple embedded real-time operating system (Embedded-RTOS). The course further looks at the principles of an Embedded-RTOS, its operation, and how real-time applications may be programmed using an Embedded-RTOS. The lectures are accompanied by a lab consisting of two parts: (i) an extensive introduction into C programming, and (ii) a hands-on lab in which students acquire the skills of designing and implementing control software for a home-brewed robot platform.	
Study Goals	The learning objectives for this course can be summarized as follows: - At the end of this course, the student should know how to employ the principles of real-time programming, on top of embedded real-time operating systems. In order to achieve this, the student must be able to illustrate the characteristics of embedded-systems programming, to formulate the principles of real-time programming and to explain the mechanisms behind Embedded-RTOSes; - The secondary goal of this course is to provide the theoretical and the experimental training, allowing the student to differentiate the principles of good embedded systems design and quantify the estimated time to achieving an embedded-software project; - At the end of this course, the student should be able to use the development environment and tool chain needed for programming the hardware used in the lab.	
Education Method	Lectures (20 hrs) + programming lab (72 hrs) + self study (48 hrs)	
Literature and Study Materials	See course website	
Books	"An embedded systems primer", David E. Simon Addison Wesley, ISBN 0-201-61569-X	
Assessment	Multiple-choice and open questions exam + programming assignments. The end mark is a combination of the lab results (60%) and the written exam (40%); both parts (exam, lab) must be passed with a minimum mark of 5. A resit will be scheduled for the exam; individual lab assignments that do not meet the quality constraints may be retried once. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Permitted Materials during Tests	None (closed-book exam)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Prof.dr. K.G. Langendoen	

CSE2430	Operating Systems	5
Responsible Instructor	Dr. J.E.A.P. Decouchant	
Responsible Instructor	T.V. Aerts	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Parts	The lab component requires hardware, which will be made available to registered participants in the course. A deposit is required for the hardware, which will be returned when the hardware is handed in.	
Course Contents	This course gives an introduction to the design and implementation of Operating Systems, the software that manages hardware and software resources on modern computing machinery. Topics discussed in class include: - function and structure of Operating Systems, system calls, interrupts, and exceptions, dual-mode execution, kernel architecture, device drivers - processes, running multiple processes, and the various algorithms for CPU scheduling and their properties - threads, inter-process communication, programming for concurrent execution and the danger of race conditions - process synchronization, critical sections, mutual exclusion, and mechanisms for these, deadlocks and solutions - memory management techniques, virtual memory, paging and page table management - protection and security, access control, security policies and mechanisms, side-channel attacks - IO and storage, programming with IO, hard disk drives, disk scheduling, SSDs, RAID - file systems, block allocation, directory structures, file systems in practice; - virtualization, emulation, virtual machines. Concepts and principles discussed in in this course are generally valid for any operating system, specific implementations like Linux are used as illustrations.	
Study Goals	Students are able to describe the fundamental problems and solutions in Operating Systems (resource management, dealing with concurrent tasks, abstracting from hardware) and apply them to practical problems. Students are able to choose and defend solutions to new systems problems. Students are able to design and implement basic operating system algorithms such as process/thread scheduling, thread synchronization, page replacement, disk IO scheduling, etc. Students are able to conduct experiments with the goal of evaluating system performance, scalability, and other non-functional requirements.	
Education Method	Lectures and Lab assignments in Linux on the Raspberry Pi. Homework to practice exam-like questions are handed out during the semester. The homework are self-assessed with an option to discuss during lectures. Additional feedback is possible during lab sessions.	
Practical Guide	Lab manual, available on Brightspace	
Books	Operating System Concepts by Silberschatz, Galvin & Gagne, 10th edition, 2021.	
Assessment	The final grade of the course consists of the following components: - Mandatory Lab (pass/fail) consisting of: --- Multiple mandatory lab assignments with individual deadlines (pass/fail) --- Return of the borrowed hardware (pass/fail) - Written exam (100%) - (optional) Lab Bonus (max 0.5 points) Final grade calculation: - Mandatory Lab failed -> 1 - Mandatory Lab passed and Exam Grade < 5.8 -> Exam Grade - Mandatory Lab passed and Exam Grade >= 5.8 -> Exam Grade + Lab Bonus Bonus: A bonus of up to a maximum of 0.5 points on the final grade may be obtainable through bonus assignments in the lab. The bonus only applies if the grade for the course without bonus is already a passing grade (>= 5.8), i.e. the bonus cannot bring a grade to a passing grade. Repair possibilities: - Mandatory Lab - Assignments: each assignment may be resubmitted and re-sign-offs may be attempted an unlimited amount of times before its corresponding deadline, as long as TA capacity permits. Based on previous years this effectively means that students who start early can do 2-4 sign-off attempts. Additionally, exactly one assignment (for which no passing result was obtained before its deadline) can be repaired. In this case, the deadline for the assignment being repaired and for all subsequent lab assignments moves by one week and normal policy applies (unlimited resubmissions and re-sign-offs under the conditions stated above). This moving of subsequent deadlines is to give the student breathing room to catch back up. - Mandatory Lab - Hardware Hand-in: failure to hand in the borrowed hardware at the scheduled moment results in a failure for the mandatory lab. An individual appointment with the instructor can be made in the quarter after the course runs to still return the borrowed hardware. Returning the hardware at such an appointment is counted as a repair for this component. - Written exam: a resit is provided in the next quarter Transferring partial results to different academic years: - Lab results (pass/fail) from a previous year can be honored if the content has not changed. Prior approval from the instructors is required for lab results to be transferred from a previous year. - Bonuses cannot be transferred between years. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Special Information	Email: os-cs-ewi@tudelft.nl . Please send any emails related to the course to this address.	
Co-Instructor	T.V. Aerts	

CSE2510	Machine Learning	5
Responsible Instructor	Dr.ir. J.H. Krijthe	
Responsible Instructor	M.A. Migut	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 4/0/0/0 lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Expected prior knowledge	CSE1100/TI1206 Object-oriented programming CSE1305/TI1316 Algorithms and Data Structures CSE1200/TI1106M Calculus CSE1205/TI1206M Linear Algebra CSE1210/TI2216M Probability Theory and Statistics	
Course Contents	The goal of the course is to acquaint students with the basic Machine Learning concepts and algorithms. Specifically, the course will cover parametric and non-parametric density estimation, linear and non-linear classification, unsupervised learning including clustering and dimensionality reduction, performance evaluation of predictive algorithms and ethical issues in machine learning.	
Study Goals	After successfully completing this course, the student is able to: - explain the basic concepts and algorithms of machine learning and their underlying statistical concepts. - implement, apply and evaluate basic ML algorithms in Python. - explain the concept of and identify (implicit) bias in data and ML algorithms.	
Education Method	The course comprises two lectures and one (four-hour) lab session per week.	
Assessment	Computer exam: 100% of the final grade. Reparation: There is one resit per year for the digital exam. The passing grade is 5.8. Bonus assignment: a written bonus assignment will be published towards the end of the course. The assignment is optional. When a passing grade is obtained for this assignment, 0.5 is added to the final grade of the exam (but only if a passing grade for the exam was obtained before the bonus is applied, and with a maximum final grade for the course of 10). Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Dr. D.M.J. Tax	

CSE2520	Big Data Processing	5
Responsible Instructor	Dr. B. Özkan	
Contact Hours / Week x/x/x/x	4/0/0/0 lecture; 4/0/0/0 lab	
Education Period	1	
Start Education	1	
Exam Period	1 2	
Course Language	English	
Course Contents	The term "Big Data" describes datasets that are either too big or change too fast or both to be processed on a single computer. Big Data Processing provides an introduction to systems and algorithms used to process Big Data. The main focus of the course is programming and engineering big data systems; initially, the course explores general programming primitives that span across big data systems and touches upon distributed systems, databases and filesystems. Then, the course examines in detail the implementation of data analysis algorithms in Spark and Flink, in the context of batch, streaming, and graph processing applications.	
Study Goals	After the end of the course, the students should be able to: - Apply basic data processing operations (filtering, folding, projecting, etc) - Explain basic techniques (vector clocks, consensus) in distributed systems - Explain basic data management techniques in distributed databases - Explain the major components of batch processing systems - Create batch algorithms for novel (unseen) practical problems - Explain the difference between iterative/non-iterative algorithms - Explain basic graph algorithms and their implementation batch processing systems - Describe in which scenarios streaming algorithms are most applicable - Apply basic streaming algorithms to practical problems	
Education Method	The teaching method consists of lectures, lab sessions and homework.	
Literature and Study Materials	Teaching materials are provided here: https://burcuku.github.io/cse2520-bigdata/	
Books	The course, by design, touches upon various current technologies; as such, there is no single source of truth. The following is an indicative list of resources where more information can be found. 1. M. Kleppmann, Designing data-intensive applications. O'Reilly Media, Inc., 2017. 2. T. Akidau, S. Chernyak, and R. Lax, Streaming systems: The what, where, when, and how of large-scale data processing. O'Reilly, 2018. 3. B. Chambers and M. Zaharia, Spark: The definitive guide. O'Reilly Media, Inc., 2017.	
Assessment	The assessment is based on practical assignments (homework and lab sessions) as well as the final exam. The assignments form 40% of the final grade and the exam 60%. The assignment grade is calculated as the mean grade of all assignments; non submitted assignments are graded with 0. Resit & Repairs The resit covers only the exam. If the final assignment grade is in between 4 and 5.7, one of the previously submitted assignments can be repaired. Partial grades (assignments or exam) cannot be transferred to subsequent years. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Permitted Materials during Tests	None	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Maximum number of participants	300	
Co-Instructor	Prof.dr.ir. D. Spinellis	
Co-Instructor	Ir. T.A.R. Overklift Vaupel Klein	

CSE2525	Data Mining	5
Responsible Instructor	Ir. S.E. Verwer	
Contact Hours / Week x/x/x/x	0/4/0/0 lecture; 0/8/0/0 lab	
Education Period	2	
Start Education	2	
Exam Period	2 3	
Course Language	English	
Expected prior knowledge	Courses considered required, i.e. students are strongly recommended to have passed all: CSE1100 Object-oriented programming CSE1305 Algorithms and Data Structures CSE1200 Calculus CSE1205 Linear Algebra CSE1210 Probability Theory and Statistics CSE2510 Machine Learning CSE2520 Big Data Processing Specific topics that are assumed as prior knowledge include: Discrete mathematics: set intersections, unions, and differences. Linear algebra: matrix multiplication, linear systems, eigendecompositions. Probability and statistics: multivariate Gaussian distribution and correlation and covariance (matrices). Programming: Java or Python programming skills. Data structures: arrays, linked lists, hash tables, and trees. These requirements are not enforced and for information purposes only. Gaps in prior knowledge will impede success in this course. Students are expected to be responsible for their own study success. We are transparent in our expectations to enable students to make the right choice for their skill set.	
Course Contents	The goal of the course is to teach students how to mine large data sets in diverse problem settings. We consider different ways to process data in order to extract knowledge in the form of profiles, anomalies, predictions, and recommendations. The course is centered around 3 core techniques: - matrix decomposition - counting and hashing - computing distances We cover the basic mathematical skills required to use these techniques effectively and you have to demonstrate mastery by developing solutions in 3 large lab assignments from scratch: - building a recommender system for profile matching - detecting anomalies in system logs - clustering data from social networks Data from these different domains often needs special (pre)processing methods to be able to apply machine learning/data mining methods. We discuss the main (pre)processing methods and their considerations in the course. Importantly, different distance measures and their effect on the mining outcome is a recurring theme. Also, special consideration is given to being able to deal with huge datasets through smart approximations. Ethical considerations of data mining are discussed. In all of these topics, the course will cover key algorithms for similar-item retrieval, frequent itemset mining, text mining, large scale clustering, collaborative filtering, dimension reduction, and graph mining.	
Study Goals	You can perform computations (by hand) required for matrix decomposition, efficient counting using hashing, complex distance computation. You are able to explain the inner workings and run by hand iterations of several key data mining algorithms: - preprocessing: SMOTE - singular value decomposition - clustering: kMeans++, CURE, DBSCAN - anomalies: Isolation Forest, PCA - counting: Frequent, Bloom Filter, CountMin Sketch - locality sensitive hashing: MinHashing - itemsets: Apriori - recommendations: Collaborative Filtering - text mining: Word2Vec - graphs: Graph Embeddings You are able to implement from scratch and apply: - matrix factorization and item-based recommender systems - count-min sketch profiles and PCA for anomaly detection - graph embeddings and girvan-newman for community detection You know important data mining concepts: - The prosecutor's fallacy, Bonferroni's principle, and data dredging - Data pre-processing methods - Important distance functions including sequence alignment - Manifolds and dimensionality reduction - Centroid, agglomerative, and density-based clustering - Anomaly detection using classifiers, neighbors, and spectral decomposition - Common sketches and sampling methods - Hashing using signatures - Frequent itemsets and association rules - Recommendation profiles and latent factor models - Explainability and bias in classification - Text embeddings and indexing - Graph embeddings and random walks You have the skills to correctly apply and make valid design choices for: - data preprocessing - computing distances - clustering methods, including speedups	

	- anomaly detection - mining sequential data - computing sketches and hashes - finding frequent patterns - making recommendations - mining text and graph data You can apply data mining algorithms to real-world challenges: - anomaly detection - recommendation system - large graph mining You can critically evaluate ethical considerations when applying data mining methods.
Education Method	The course comprises two lectures and one (four-hour) lab course per week. There are three required lab assignments performed in pair of students. There are optional home-work exercises to practice the taught skills. TAs will be available for questions during the lab hours.
Literature and Study Materials	The reading material comes from the book "Mining Massive Datasets" by Anand Rajaraman, Jure Leskovec, and Jeffrey Ullman, and the book "Data Mining: The Textbook" by Charu Aggarwal. Additional reading material in the form of scientific papers will be provided. Guest lecturers will demonstrate usage of real-world data mining systems. Any information from guest lectures is optional.
Assessment	All material from non-guest lectures provided during the course is part of the study material. Each lab assignment counts for 10% of the final graded and will be graded using guided and checked peer reviewing. A digital exam counts for 70% of the final grade. Reparation: There is one resit per year for the digital exam. The passing grade for the exam is 5.8.
Permitted Materials during Tests	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4). Pen, ruler, calculator. No other materials are allowed.
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/
Co-Instructor	Dr. A. Anand

CSE2530	Computational Intelligence	5
Responsible Instructor	Dr. F.A. Oliehoek	
Responsible Instructor	L. Cavalcante Siebert	
Responsible Instructor	Dr. P.K. Murukannaiah	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Course Contents	This course introduces artificial intelligence techniques that focus on learning, optimization and self organisation. We cover numerical techniques that can be grouped as computational intelligence including: - Neural networks - Evolutionary Systems - Swarm intelligence - Reinforcement learning	
Study Goals	After completing this course, you should be able to: - LO1: Position Computational Intelligence (CI) as a field in AI - LO2: Explain concepts such as problem space, fitness, and optimization - LO3: Understand the working and limitations of CI techniques - LO4: Select the most appropriate CI technique for a given problem - LO5: Apply Reinforcement Learning techniques for a given problem - LO6: Apply different types of neural networks for a given problem - LO7: Apply evolutionary and swarm techniques to solve given problems	
Education Method	Lectures + practicum	
Literature and Study Materials	In this course, we use different material for each topic: - For introduction to CI, neural networks, and genetic algorithms: Book "Artificial intelligence - A guide to intelligent systems, third edition" by Michael Negnevitsky - For reinforcement learning we use the online book "Reinforcement learning: An introduction" by Sutton and Barto. Available at: http://incompleteideas.net/sutton/book/ebook/the-book.html - For Neural Networks we use the book: "Deep learning" by Goodfellow, Bengio and Courville. Available at: https://www.deeplearningbook.org/ - For other topics, we use scientific articles. If a reader is provided, this also is part of the study material.	
Assessment	The final grade of the course consists of the following components: - Written Examination (50%) - Group Practical Assignments (50%) Final grade calculation: $(0.5 * \text{Written Examination} + 0.5 * \text{Group Practical Assignments})$ Both the written exam and the group assignments need to be completed with a minimum of a 5.0 Practicum and exam are PARTS of you final grade and will not considered as separate grades in OSIRIS. Only your final grade for a year will be registered. For the exact calculation of the final grade, please see the sheets of the introductory lecture. Resit & Repair: The written exam has one resit in the same academic year. If the final grade of the group (lab) assignments is between 4.0 and 5.7 , the group can choose one of the three assignments for resubmission. The group can get any grade for the repair assignment, but the final lab grade will not exceed 6.0. Group members can decide if they want to join the repair attempt or not. The deadline for resubmitting the chosen assignment will be about two weeks after the final exam. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Permitted Materials during Tests	Calculator, pen and paper	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Tags	Artificial intelligence	
Co-Instructor	Ir. T.A.R. Overklift Vaupel Klein	

Year	2023/2024
Organization	Electrical Engineering, Mathematics and Computer Science
Education	Bachelor Computer Science and Engineering

Bachelor 3rd year TI 2023

Prerequisites	Please note! The entry requirements are mentioned in the regulations https://www.tudelft.nl/en/student/eemcs-student-portal/education/rules-regulations. If there are differences between what is mentioned here and what is mentioned in the regulations, that what is mentioned in the regulations is binding. To be able to participate in CSE3000 Research Project you must have passed all the courses of the first year + the first semester of the second year + the CSE2000 Software Project if you are from cohort 2018 or later. If you are from cohort 2017 or before, you must have passed all the courses of the first and the second year of the bachelors programme. If you didnt pass the required courses/projects and think that not participating in the project will result in unduly study delay you may hand in an application for an entry requirements exemption. You will receive more information during assemblies and by e-mail.
Introduction 1	Registration for exams and registration for courses and projects All elective courses and projects in year 3 require registration through Osiris, which will be open in Q2. Registration for the end project will be done through ProjectForum. For the exams of all the courses you have to register at least two weeks before the day that they take place. More information is available on: https://www.tudelft.nl/en/student/eemcs-student-portal/education/registration-courses-and-examinations. If you decide after your register that you dont want to participate in the project or exam dont forget to cancel your registration.
Introduction 2	In the third year you will follow a minor. For more information you can go to https://www.tudelft.nl/en/education/programmes/minor/. If after your Bachelors you want to do a Masters now is the time to orientate yourself. Keep an eye on the announcements on Brightspace about the master events.

CSE3000	Research Project	15
Responsible Instructor	M.A. Migut	
Contact Hours / Week x/x/x/x	0/0/0/4	
Education Period	4	
Start Education	4	
Exam Period	4	
Course Language	English	
Course Contents	This course comprises an individual research project on a selected topic within computer science, supervised by an active researcher in the field. It should result in a scientific paper (of at most 8 two-column or 14 single-column pages) describing the research question, results, conclusions and reflection. The course is completed with a conference day with poster presentations by all students.	
Study Goals	1. The student has an in-depth understanding of a selected area of computer science beyond that available from other undergraduate courses, at or near state of the art. 2. The student can execute a small research project and report about it, which includes that they are able to a. search and interpret scientific literature, critically assess scientific work, and the relevance for their own research project, b. clearly (re)formulate and motivate a provided research question, c. execute an appropriate research method, d. write a research plan with help of a given template, e. use, explain and justify adequate research and design methodologies, f. apply theory to the performed project, g. use techniques for interpretation and verification and base conclusions on results, h. answer the research question in a clear way, i. write a scientific paper with help of a given template, k. write a review on research papers of other students, l. criticize his/her own results, m. present the performed research, n. discuss and reflect on reproducibility and integrity of the performed research.	
Education Method	During the course, the students may work and discuss in groups. At least once a week they meet with their supervisor. Peer feedback as well as feedback from the responsible professor is organized on essential steps in the research project: 1. A research proposal 2. A midterm feedback meeting 3. A scientific paper, describing the research (question, results and conclusions) 4. A poster summarizing the research Additionally, support and/or lectures are given regarding research methodology, finding and evaluating scientific work, scientific writing and presenting, and responsible computer science.	
Prerequisites	Students get permission to participate in the Research Project once they have completed 1) all courses of the first year, 2) all courses of the first half of the second year of the bachelor programme, and 3) the Software Project (CSE2000).	
Assessment	First, a pass is needed for the Information Literacy assignment. The final grade for your project is then computed based on a weighted sum of four main components: 1. Content (40%) Assessment of related scientific work: Student shows ability to critically assess (previous) scientific work. Research/design methodology: The student is able to use, explain and justify adequate research and design methodologies. Scientific method: The chosen research approach is adequate and the work is reliable. Interpretation of results: Student uses techniques for interpretation and verification and base conclusions on results. Responsible engineering: The student adequately reflected on the reproducibility and integrity of the performed research. 2. Quality of presentation on paper (30%) Argumentation: Student expresses argumentation coherently and concisely. Structure and references: Organisation of the content is satisfactory. References are well documented. Motivation: The motivation of the work is sufficiently clear. Quality of language: The student uses correct spelling and grammar. 3. Process (20%) Planning: Student adequately managed the project. Independence: Student took initiative and needed little guidance. Peer feedback and communication: Student interacted well with peers/supervisor(s) and gives clear feedback to peers. 4. Poster Presentation (10%) Presentation/poster content and performance: Student demonstrates extensive knowledge of the subject, visuals are clear. The final examination will take place in week 10 and the grade is agreed upon by the responsible professor and an independent examiner, using input from the supervisor. There is no possibility for a repair option. If necessary, you can do a new project at the next iteration. This course is offered in Q2 and Q4. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4).	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via ProjectForum in weeks 2.8-2.9 for Q4 and weeks 4.6-1.3 for Q2. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Special Information	Upon special request, it is possible for students to research a self-proposed topic. For this, they will need to submit a research proposal (including motivation, relevant literature and about five novel research questions) to the course organization via an email to rp-cs-ewi@tudelft.nl before February 1 for Q4 and before September 1 for Q2. The proposal should include suggestions for relevant responsible professors or research groups. If the proposal is permitted, it will also be open for other students to join, and it will be treated like any other proposed topic in the group assignment, which means it is not guaranteed to be selected for execution unless the group is complete. Upon further special request, it is possible to do this research project as a research internship project at another university under the following conditions: * All conditions for self-proposed topics as described above apply.	

- * The research takes place within 10 weeks (from writing the research plan to presenting your results), conform the regular schedule of the course.
- * The student(s) find a remote supervisor, arrange and pay for their own travel and housing.
- * The student(s) perform(s) a peer review during the period.
- * The student(s) make(s) a suitable assignment on scientific writing during the period.
- * The final examination will take place in week 10 at the TU Delft (if necessary through teleconference) and the grade is agreed upon by the remote supervisor, the TUD responsible professor and the TUD examiner.

Co-Instructor

Prof.dr. M.M. de Weerd

CSE3100	Functional Programming	5
Responsible Instructor	Dr. J.G.H. Cockx	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Expected prior knowledge	In order to take this course, you should know how to program in at least one (not necessarily functional) language, for example Java as taught in Object Oriented Programming (CSE1100). You should also be familiar with basic principles in the design of programming languages, e.g. by taking the course Concepts of Programming Languages (CSE2120).	
Course Contents	Functional programming is a style of programming where programs are constructed by defining and applying functions. In pure functional code, the output of a function only depends on the value of its arguments, so the same expression always produces the same result. This is in sharp contrast to imperative and object-oriented languages, where every statement can have side effects that influence the results of other statements. Apart from the absence of side-effects, another important feature of most functional programming languages are their powerful type systems. By modeling the different classes of data used in a program, types can be used to structure programs and detect many common errors that would otherwise go unnoticed. Certain languages with so-called dependent type systems can even specify any imaginable functional property in the type of a program, guaranteeing that the program works as it is supposed to before it is ever executed. Why study functional programming? Functional programming allows you to model the real world at a high level of abstraction in order to develop software that is clear, concise, and correct. By learning functional programming, you will also learn new ways to think about programs that make you a better programmer in any programming language. Functional programming languages are becoming increasingly commonplace, and other mainstream languages are integrating more functional features (lambdas, higher-order functions, pattern matching, immutable data, ...) Pure functional programming makes it easy to reason about program correctness by using techniques such as equational reasoning, property-based testing, and formal verification. Static typing and immutability allow you to quickly change and refactor an existing program with confidence that the result is correct. Functional programming languages such as Haskell and Agda include many recent developments from the research on programming languages, which usually take 10 to 30 years to make their way into mainstream languages. By learning them you will get an idea how the future of programming languages might look like.	
Study Goals	In the first part of this course, you will learn how to program in Haskell, a purely functional programming language. Concretely, you will learn... - how to write programs that can manipulate and return functions as first-class data. - how to define and manipulate your own data structures, by using algebraic datatypes, pattern matching, and recursion. - how to automatically test important properties of your programs with the QuickCheck tool. - how to avoid code duplication and write code at a higher level of abstraction by using type parameters and type classes. - how to avoid unnecessary computation and program with infinite data structures by relying on lazy evaluation. - how to distinguish between pure and impure computations, and use monads to write programs that make this difference explicit. In the second part of this course, you will learn how to use Agda, another purely functional programming language that is similar to Haskell but has a more powerful dependent type system. In this part, you will learn... - how to express logical properties as types through the Curry-Howard correspondence. - how to develop Agda programs interactively with the typechecker and detect errors as you are writing the program. - how to enforce invariants of your programs at the type level by using indexed datatypes and dependent pattern matching. - how to formally prove properties of purely functional programs by using the identity type and equational reasoning.	
Education Method	The course work consists of the following activities: 2 instruction sessions per week, typically consisting of 45 minutes of lecture followed by 45 minutes of tutorials. 4 hours of exercise classes per week. - A large project to be implemented in Haskell or Agda.	
Literature and Study Materials	The official book for the course is Programming in Haskell (http://www.cs.nott.ac.uk/~pszgmkh/pih.html) by Graham Hutton (second edition). The online book Learn You a Haskell for Great Good! (http://learnyouahaskell.com/chapters) by Miran Lipovaca is free and provides an excellent introduction to many aspects of Haskell that are covered in the course. For part of the course on Agda, reading materials will be distributed via Brightspace.	
Assessment	The assessment for this course consists of two parts: - The course project (50%) - The final written exam (50%) To pass the course, each of these grades (course project and final exam) should be 5.0 or higher, and the final grade should be 5.8 or higher (and will be rounded to the nearest half grade point). The regular exam is held in April in the exam period. There is a resit for the exam during Q4. There is also an opportunity to resubmit an improved version of the project during Q4. The maximum grade for resubmissions of the project is 6.0.	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Dr. B.P. Ahrens	

CSE3130	Introduction to Quantum Computer Science	5
Responsible Instructor	G.S. Vardoyan	
Contact Hours / Week x/x/x/x	0/0/2/0 lecture; 0/0/2/0 tutorial; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Expected prior knowledge	CSE1205 Linear Algebra, CSE1100 OO programming, CS1210 probability theory and statistics, CS1200 Calculus	
Course Contents	This course introduces the basics of quantum information from a computer science perspective. Emphasis is given to 1) the actual programming of quantum computers and 2) understanding rigorously their power and limitations from a complexity point of view. Taking this class will also prepare students for the master level courses from QuTech academy or QIST master program.	
Study Goals	After the course, the students will be able to: 1. Manipulate single qubits and predict the measurement outcomes after a sequence of quantum gates. 2. Manipulate multiple qubits and predict the measurement outcomes after a sequence of quantum gates. 3. Decompose a gate into a sequence of gates from a universal discrete set. 4. Describe basic quantum algorithms and primitives such as Bernstein-Vazirani and the quantum Fourier transform. This includes predicting the success probability of the algorithm (if applicable) and explaining the complexity of the algorithm. 5. Independently describe standard textbook material beyond the class content such as Shors algorithm for factoring.	
Education Method	The course will follow a blended approach. Each week, the students will be asked to 1) read the lecture notes in advance of the theory lectures and 2) attempt to solve the weekly programming assignments before the practicums. Class time will be divided into theory lectures and practicums which may include short tutorials. The theory lectures will cover the main concepts of the week. In the practicums we will discuss the theory behind the programming assignments and jointly solve some challenging parts of the assignments or other exercises.	
Assessment	There will be weekly theory and programming assignments. The grade of the course will be the grade of the final exam.	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). This course is an exception to the rule and is also open for B-TW and B-EE students. Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Tags	Algebra Algorithmics Information & Communication Linear Algebra Programming Quantum	
Co-Instructor	Prof.dr. A. van Deursen	

CSE3210	Collaborative Artificial Intelligence	5
Responsible Instructor	Dr. P.K. Murukannaiah	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Course Contents	Traditionally, Artificial Intelligence (AI) applications are realized as centralized solutions, where a single entity, e.g., an organization, collects massive amounts of data from end users to provide a suite of AI-powered services. The ethical concerns such centralized AI solutions pose (e.g., fairness, accountability, transparency, and privacy) are increasingly evident. This course introduces collaborative artificial intelligence, which facilitates interactions among intelligent agents, including humans and AI agents, to realize decentralized AI applications. The topics covered include: * Co-active design of decentralized AI systems * Negotiation among autonomous agents * Agent communication and interaction protocols * Agent coordination mechanisms	
Study Goals	Upon completion of this course, a student would have learned to: * Compare centralized and collaborative AI paradigms. * Apply co-active design to solve a collaborative AI problem. * Describe the principles of automated negotiation for facilitating agent cooperation. * Describe the conceptual underpinnings of agent interaction protocols. * Describe the conceptual underpinnings of agent coordination mechanisms. * Create an automated negotiating agent in the Genius Web platform.	
Education Method	Lectures + Practicum	
Assessment	* Written Examination (50%) * Group Assignments (50%) The exam and the assignments are both intermediate results, and will not count separately for the next academic year. Only the final grade will remain. For the exact calculation of the final grade, see the introductory lecture. Resit & Repair The written exam has one resit in the same academic year. The group assignments (projects) do not have a resit. The projects take place during the quarter. A repair option can be provided only for students failing the projects. The maximum grade achievable for the projects with the repair option is 6.0. The repair option typically includes redoing parts of the project that are found to be insufficient in the original submission. The projects during the quarter involve group work, but the repair option may be offered solo.	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Prof.dr. C.M. Jonker	
Co-Instructor	M.L. Tielman	
Co-Instructor	L. Cavalcante Siebert	

CSE3220	Computer Security	5
Responsible Instructor	Dr. K. Liang	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Course Contents	This course aims to include the following topics: 1. Introduction to Computer Security - o Overview concepts of computer security - o Threats and adversarial models - o Security principles and technologies 2. System Security - o Authentication and authorization - o Database security 3. Network Security - o Secure network protocols - o Network attacks and defenses - o Firewalls and intrusion detection systems - o Internet Security 4. Applied Cryptography - o Symmetric and asymmetric encryption - o Hash functions and message digests - o Digital signatures and certificates - o Key exchange 5. Advanced topics - o AI and security - o Risk management - o Security policies and procedures Recommended textbook: Computer Security: Principles and Practice (4th edition), by Lawrie Brown and William Stallings (Pearson Education) https://www.pearson.com/en-gb/subject-catalog/p/computer-security-principles-and-practice-global-edition/P200000005478	
Study Goals	This course is an introduction to the fundamental concepts of computer security. It covers the basic principles and techniques of securing computer systems, including systems security, network security, database security, cryptography and authentication, as well as selected advanced security topics. It also covers security risks, threat models, vulnerabilities, and security policies. Learning Outcomes: By the end of this course, students should be able to: - Understand the principles and concepts of security and privacy. - Apply core security principles/properties to secure systems. - Analyze security risks and attacks in authentication and network. - Apply secure tools and algorithms into information security context. - Analyze security issues and propose security solutions.	
Education Method	Lectures combined with assignments.	
Assessment	Two assignments (20%) and a 3-hour written exam (80%). Resit & Repair For the exam a resit will be offered. For the two assignment a repair will be offered. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Co-Instructor	Ir. S.E. Verwer	

CSE3230	Algebra and Cryptography	5
Responsible Instructor	Dr. K. Liang	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Course Contents	This course will be replaced by CSE3220 Computer Security from 2023-2024 onwards. Students who need to resit Algebra & Cryptography will be offered a final exam (and a resit) opportunity in academic year 2023-2024 or are encouraged to follow CSE3220 Computer Security. This course may include the following topics: 1. Introduction to Computer Security - o Overview concepts of computer security - o Threats and adversarial models - o Security principles and technologies 2. System Security - o Operating Systems security and access control - o Authentication and authorization - o Database security - o Software security and testing - o Malware detection and prevention techniques 3. Network Security - o Secure network protocols - o Network attacks and defenses - o Firewalls and intrusion detection systems - o Internet Security - o Web and c-loud security 4. Applied Cryptography - o Symmetric and asymmetric encryption - o Hash functions and message digests - o Digital signatures and certificates - o Key exchange 5. Advanced topics - o AI and security - o Mitigation strategies - o Risk management - o Security policies and procedures Recommended textbook: Computer Security: Principles and Practice (4th edition), by Lawrie Brown and William Stallings (Pearson Education) https://www.pearson.com/en-gb/subject-catalog/p/computer-security-principles-and-practice-global-edition/P200000005478	
Study Goals	This course is an introduction to the fundamental concepts of computer security. It covers the basic principles and techniques of securing computer systems, including systems security, network security, software security, cryptography and authentication, as well as selected advanced security topics. It also covers security risks, threat models, vulnerabilities, and security policies. Learning Outcomes: By the end of this course, students should be able to: - Understand the principles and concepts of security and privacy. - Apply core security principles/properties to secure systems. - Analyze security risks and attacks in authentication and network. - Apply secure tools and algorithms into information security context. - Analyze security issues and propose security solutions.	
Education Method	lectures combined with assignments	
Assessment	For this course: a 3-hour written exam (100%).	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	Dr. K. Liang	

CSE3300	Algorithms for NP-Hard problems	5
Responsible Instructor	Dr. N. Yorke-Smith	
Contact Hours / Week x/x/x/x	0/0/4/0 lecture; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Expected prior knowledge	Required: CSE2310 Algorithm Design CSE1300 Reasoning and Logic CSE1210 Probability Theory and Statistics Recommended: CSE2530 Computational Intelligence Related: CSE2315 Automata, Computability and Complexity CSE2510 Machine Learning	
Course Contents	Combinatorial optimisation problems such as scheduling or optimally playing a board game are often NP-hard. Most of the algorithms in earlier courses run in polynomial time and cannot be directly used for solving NP-hard problems. In this course you will learn some new algorithms, and use these, but also concepts from earlier courses on algorithm design, to solve NP-hard problems.	
Study Goals	The students are able to position exact and approximate algorithms for combinatorial search problems within AI. The students are able to apply best-first and A* search to combinatorial problems. The students are able to improve exhaustive combinatorial search by applying simple pruning and propagation techniques. The students are able to apply heuristically-guided Monte Carlo Tree Search for the example of playing a board game. The students are able to apply Monte Carlo simulations to planning problems. The students are able to improve simulation-based planning with statistical methods. The students are able to explain the dynamic programming design principle in simple stochastic settings.	
Education Method	The course consists of 2 hours of lectures and 4 hours of lab per week. The lectures are used to introduce NP-hard problems and algorithmic techniques to deal with them. The labs are used to try the (algorithmic) concepts. The course has graded, formative programming assignments. In the assignments you are asked to apply the technique to (a variant of) such a problem by implementing (part of) the algorithm and running it on a set of problem instances. We expect to have three such programming assignments, taking about 8-10 hours each, and performed in small groups.	
Assessment	The final grade is computed based on a weighted average of the grade for the programming assignments and the final written exam, as follows: A = unrounded, unweighted arithmetic mean of assignment grades E = unrounded exam grade $C = 0.3 * A + 0.7 * E$ C is rounded to the nearest 0.1 A resit exam will be available, whereupon $E = \max(E_{\text{exam}}, E_{\text{resit}})$. For each assignment i, students who receive a grade between 4.0 and 5.7 may re-submit the assignment, whereupon $A_i = \min(6.0, \max(A_{i_original}, A_{i_resubmission}))$. Students who receive a grade on assignment i outside these bounds may not re-submit the assignment, i.e. if $A_i < 4.0$ or $A_i > 5.7$. To pass the course in terms of grades, the student must satisfy the conjunction of these constraints: A > 5.0 E > 5.0 C > 5.7	
Enrolment / Application	Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4) Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Special Information	Course email address: algnphard-cs-ewi@tudelft.nl	
Elective	Yes	
Tags	Algorithmics Artificial intelligence Group work Logics Mathematics Modelling Programming Projects	
Co-Instructor	Dr. E. Demirovi	
Co-Instructor	Dr. A. Lukina	

CSE3500	Human Computer Interaction	5
Responsible Instructor	M.L. Tielman	
Contact Hours / Week x/x/x/x	0/0/2/0 lecture; 0/0/2/0 tutorial; 0/0/4/0 shared lab	
Education Period	3	
Start Education	3	
Exam Period	3 4	
Course Language	English	
Course Contents	Traditionally interaction with computer systems moved from text-based (e.g., terminals) to graphical user interfaces (GUI). These evolved to a wider range of platforms than just desktop computers, to include tablets, smart phones, and even robots. As technology systems (e.g., expert systems, recommender systems) increase in complexity, a standard graphical user interface (GUI) is often not sufficient to harness their power. Moreover, not all users are the same, and to design a system to interact well with a specific user group, taking them into account from the start is key. This course will cover four aspects of HCI systems: a) Identifying functional and non-functional requirements. b) Design of interactive and adaptive systems. c) Prototyping of these systems. d) User-centered Evaluation of these systems.	
Study Goals	After this course, students will be able to: Objective 1. Understand the concepts and theories of human-computer interaction. Furthermore, they will be able to explain basic concepts from the relevant theories in cognitive and social sciences. Objective 2. Illustrate and compare methods of "user-centered engineering (UCE)", involving future users in various stages of the design and development process. Objective 3. Apply "user-centered" methods appropriately, as a team; as part of design, development, and evaluation of software systems. Objective 4. Apply ethics procedures for user studies (e.g., privacy, consent). Objective 5. Describe the perspectives and values of multiple stakeholders at different stages of the user-centered engineering process (requirements, design, and evaluation).	
Education Method	This course consists of 8 2-hour lectures and 16 2-hour labs, of which 8 are shared and 8 are specifically for HCI. In the HCI specific labs you will work on assignments related to the tasks necessary for the group work. In the rest of the time (including the shared labs) you can work on the group work under supervision of the TA's who are available for feedback. Starting from Week 1, students form groups and work on a project, to be handed in in week 9. Students are expected to work 6 hours per week (each) on the project assignment. Expected workload is 16 hours for attending lectures, 25 hours of reading study material and preparing lectures, 32 hours for attending lab sessions, 59 additional hours for group assignment, and 8 hours for exam preparation (total 140 hours).	
Literature and Study Materials	Book: David Benyon. Designing User Experience: A guide to HCI, UX and interaction design (4th Edition) Slides and Articles will be available on Brightspace	
Assessment	Mandatory assessments: - Group-work assessment (LO2-LO5) (4-6 people): 50% (w.3.9) (Based on written report) - Exam (LO1): 50% (w. 3.10) Students will need at least a 5 on group-work and exam, and average of both of at least 5.8 to complete the course. Groups will peer-review contribution to group work on a number of aspects. In case of large deviations this will be used to adjust individual grades for the group work. Formative assessments: - TA's will be available during the labs for formative feedback. TA's are also available to record and resolve any issues with group dynamics. Resit & Repair: There will be a resit for the exam in Q4. The written report can be resubmitted if the final grade is between 4.0 and 5.7. The repair grade will not exceed 6.0. Group members can decide if they want to join the repair attempt or not. Disclaimer: information may change depending on unforeseen circumstances or measures (see: TER Art 29, sub 4)	
Enrolment / Application	Courses of the bachelor CSE are only open to students with an active and valid registration for the B-CSE programme or an EEMCS bridging programme (CS, ES, CE) in accordance with article 8 of the Implementation Regulation in the Dutch EEMCS Teaching and Exam Regulations (article 5 in the English translation for CSE). Registration for this course takes place via Osiris, separately from the exam registration. For more information, see: https://www.tudelft.nl/en/student/faculties/eemcs-student-portal/education/registration-courses-and-examinations/	
Co-Instructor	U.K. Gadiraju	

T.V. Aerts

Department	Computer Science & Engineering-Teaching Team
Telephone Room	+31 15 27 83830 28.2.E340

Dr. B.P. Ahrens

Department	Programming Languages
Telephone	+31 15 27 89972

Dr. A. Anand

Department	Web Information Systems
Room	28.3.W660

I.C. de Bruin

Department	Computer Science & Engineering-Teaching Team
-------------------	--

Dr. J.W. Böhmer

Department	Algorithmics
Telephone	+31 15 27 86110

L. Cavalcante Siebert

Department	Interactive Intelligence
Telephone Room	+31 15 27 87606 28.4.W900

Dr. J.G.H. Cockx

Department	Programming Languages
Room	28.3.E340

Dr. J.E.A.P. Decouchant

Department	Distributed Systems
Telephone	+31 15 27 83804

Dr. E. Demirovi

Department	Algorithmics
Telephone Room	+31 15 27 87177 28.4.E400

Prof.dr. A. van Deursen

Unit	Elektrotechn., Wisk. & Inform.
Department	Software Technology
Telephone Room	+31 15 27 82486 28.1.W600

S. Dumani

Department	Algorithmics
Telephone Room	+31 15 27 88941 28.2.W600

Prof.dr. E. Eisemann

Unit	Elektrotechn., Wisk. & Inform.
Department	Computer Graphics and Visualisation
Telephone Room	+31 15 27 82528 28.5.W860

A.W.F.A.M. Elnouty

Unit	Elektrotechn., Wisk. & Inform.
-------------	--------------------------------

Department	Computer Science & Engineering-Teaching Team
Telephone Room	+31 15 27 86989 28.2.E260

Dr. E. Emsiz

Unit	Elektrotechn., Wisk. & Inform.
Department	Applied Probability
Telephone Room	+31 15 27 84579 36.HB 07.060

M. Finavaro Aniche

Unit	Elektrotechn., Wisk. & Inform.
Department	Software Engineering
Telephone Room	+31 15 27 84211 28.4.W660

U.K. Gadiraju

Department	Web Information Systems
Telephone Room	+31 15 27 87271 28.3.W740

Dr. J.C. van Gemert

Unit	Elektrotechn., Wisk. & Inform.
Department	Pattern Recognition and Bioinformatics
Telephone Room	+31 15 27 88434 28.6.E340

S. Hugtenburg

Unit	Elektrotechn., Wisk. & Inform.
Department	Computer Science & Engineering-Teaching Team
Telephone Room	+31 15 27 86845 28.2.E400

Dr. H.S. Hung

Unit	Elektrotechn., Wisk. & Inform.
Department	Pattern Recognition and Bioinformatics
Telephone	+31 15 27 85697

E. Isufi

Department	Multimedia Computing
Telephone Room	+31 15 27 89089 28.5.E220

Prof.dr. C.M. Jonker

Unit	Elektrotechn., Wisk. & Inform.
Department	Interactive Intelligence
Telephone Room	+31 15 27 81315 28.6.W600

Dr. A. Katsifodimos

Unit	Elektrotechn., Wisk. & Inform.
Department	Web Information Systems
Telephone Room	+31 15 27 87247 28.4.E080

M. Khosla

Department	Multimedia Computing
-------------------	----------------------

N. de Kleijn

Department	Statistics
Telephone Room	+31 15 27 84114 36.HB 06.260

J.J. Koorn

Department	Onderwijs en Studentenzaken
-------------------	-----------------------------

Department	Onderwijs en Studentenzaken
-------------------	-----------------------------

C. Koutras

Unit	Elektrotechn., Wisk. & Inform.
Department	Web Information Systems
Telephone	+31 15 27 87331
Room	28.4.W920

I.D. van Kreveld

Department	Computer Science & Engineering-Teaching Team
Room	28.2.E400

Dr.ir. J.H. Krijthe

Department	Pattern Recognition and Bioinformatics
Telephone	+31 15 27 88433
Room	28.5.E820

Prof.dr.ir. F.A. Kuipers

Unit	Elektrotechn., Wisk. & Inform.
Department	Networked Systems
Telephone	+31 15 27 81347
Room	28.2.E160

J.E. van der Laan

Unit	Elektrotechn., Wisk. & Inform.
Department	Onderwijs en Studentenzaken
Telephone	+31 15 27 83506
Room	36.LB 02.200

Prof.dr.ir. R.L. Lagendijk

Unit	Elektrotechn., Wisk. & Inform.
Department	Cyber Security
Telephone	+31 15 27 83731
Room	29.02.250

Prof.dr. K.G. Langendoen

Unit	Elektrotechn., Wisk. & Inform.
Department	Embedded Systems
Telephone	+31 15 27 87666

Dr. K. Liang

Department	Cyber Security
Telephone	+31 15 27 87656

Dr. C. Lofi

Unit	Elektrotechn., Wisk. & Inform.
Department	Web Information Systems
Telephone	+31 15 27 87252
Room	28.3.W780

Dr. A. Lukina

Department	Algorithmics
Telephone	+31 15 27 82888

Dr. R. Marroquim

Unit	Elektrotechn., Wisk. & Inform.
Department	Computer Graphics and Visualisation

Telephone Room	+31 15 27 81988 28.5.W800
---------------------------	------------------------------

J.A. Martinez Castaneda

Unit Department	Elektrotechn., Wisk. & Inform. Multimedia Computing
Telephone Room	+31 15 27 85445 28.5.E060

M.A. Migut

Unit Department	Elektrotechn., Wisk. & Inform. Web Information Systems
Telephone	+31 15 27 88543

M.L. Molenaar

Department	Computer Graphics and Visualisation
Telephone	+31 15 27 87821

F. Mulder

Unit Department	Elektrotechn., Wisk. & Inform. Computer Science & Engineering-Teaching Team
Telephone Room	+31 15 27 83184 28.2.E340

Dr. P.K. Murukannaiah

Department	Interactive Intelligence
Telephone Room	+31 15 27 82291 28.4.W900

Drs. P.R. van Nieuwenhuizen

Unit Department	Elektrotechn., Wisk. & Inform. Numerical Analysis
Telephone	+31 15 27 88036

Dr. F.A. Oliehoek

Unit Department	Elektrotechn., Wisk. & Inform. Interactive Intelligence
Telephone Room	+31 15 27 87494 28.4.W600

Ir. T.A.R. Overklift Vaupel Klein

Department	Computer Science & Engineering-Teaching Team
Telephone Room	+31 15 27 82226 28.2.E360

Dr. A. Panichella

Unit Department	Elektrotechn., Wisk. & Inform. Software Engineering
Telephone Room	+31 15 27 89306 28.1.W800

P. Pawelczak

Unit Department	Elektrotechn., Wisk. & Inform. Embedded Systems
Telephone Room	+31 15 27 87491 28.2.W680

Dr. J.S. de Pinho Gonçalves

Unit Department	Elektrotechn., Wisk. & Inform. Pattern Recognition and Bioinformatics
Telephone Room	+31 15 27 83418 28.6.W900

C.B. Poulsen

Unit	Elektrotechn., Wisk. & Inform.
Department	Programming Languages
Room	28.3.E400

Dr.ing. S. Proksch

Department	Software Engineering
Telephone	+31 15 27 87215

Dr. O.E. Scharenborg

Unit	Elektrotechn., Wisk. & Inform.
Department	Multimedia Computing
Telephone	+31 15 27 83245
Room	28.5.E240

Dr.ing. M. Shahraki Moghaddam

Department	Computer Engineering
-------------------	----------------------

Dr. M. Skrodzki

Department	Computer Graphics and Visualisation
Telephone	+31 15 27 85578
Room	28.5.W560

Department	Computer Graphics and Visualisation
Telephone	+31 15 27 85578
Room	28.5.W560

Ing. A.M.J. Slats

Unit	Elektrotechn., Wisk. & Inform.
Department	Electrical Engineering Education
Telephone	+31 15 27 88787
Room	36.LB 01.260

Dr. C.O. Smet

Department	Statistics
Telephone	+31 15 27 88666
Room	36.HB 06.260

Dr. M.T.J. Spaan

Unit	Elektrotechn., Wisk. & Inform.
Department	Algorithmics
Telephone	+31 15 27 81102
Room	28.4.E340

Prof.dr.ir. D. Spinellis

Department	Software Engineering
Room	28.1.W900

Dr. D.M.J. Tax

Unit	Elektrotechn., Wisk. & Inform.
Department	Pattern Recognition and Bioinformatics
Telephone	+31 15 27 84232
Room	28.5.W860

M.L. Tielman

Unit	Elektrotechn., Wisk. & Inform.
Department	Interactive Intelligence
Telephone	+31 15 27 87516
Room	28.4.W580

G.S. Vardoyan

Department	Quantum Computer Science
-------------------	--------------------------

Ir. S.E. Verwer

Unit	Elektrotechn., Wisk. & Inform.
Department	Cyber Security
Telephone	+31 15 27 88435
Room	28.6.E100

Ir. O.W. Visser

Unit	Elektrotechn., Wisk. & Inform.
Department	Computer Science & Engineering-Teaching Team
Telephone	+31 15 27 86328
Room	28.2.E300

H. Wang

Unit	Elektrotechn., Wisk. & Inform.
Department	Multimedia Computing
Telephone	+31 15 27 88847
Room	28.5.E260

Q. Wang

Department	Embedded Systems
Room	28.2.W700

Prof.dr. M.M. de Weerd

Unit	Elektrotechn., Wisk. & Inform.
Department	Algorithmics
Telephone	+31 15 27 84516
Room	28.4.E100

Dr. N. Yorke-Smith

Unit	Elektrotechn., Wisk. & Inform.
Department	Algorithmics
Telephone	+31 15 27 83895
Room	28.4.E220

Prof.dr. A.E. Zaidman

Unit	Elektrotechn., Wisk. & Inform.
Department	Software Engineering
Telephone	+31 15 27 84385
Room	28.1.W860

Dr. X. Zhang

Department	Pattern Recognition and Bioinformatics
Room	28.6.E320

Dr. B. Özkan

Department	Software Engineering
Room	28.1.W660